

第一节 概述

1. 传统机械产品设计过程

传统的手工设计手段：

笔+图板+笔擦+三角板+丁字尺+计算器等绘图和计算工具

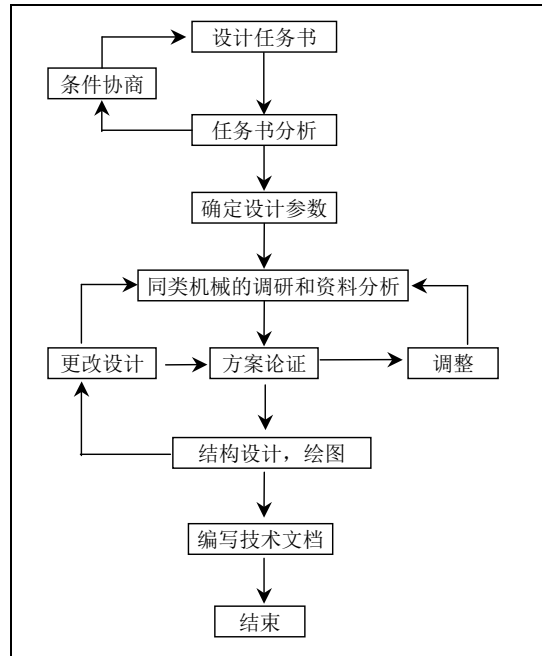


Fig1 传统的机械设计过程

传统机械设计过程的特点为：

- (1) 它的每一个环节都是依靠设计者用手工方式来完成。
- (2) 设计者大部分时间和精力耗费在绘图上，难以对整机的问题进行深入研究。
(绘图工作约占设计时间的 70%左右)

2. 计算机辅助设计 (CAD)

CAD 技术思想起源于 20 世纪 50 年代末,美国麻省理工学院 Sutherland 博士的研究工作。40 多年来, CAD 技术已发展成为以“计算机技术”和“计算机图形学”为技术基础,并融合各工程学科知识(即各应用领域中的设计理论与方法)的一种高新技术。

何为 CAD?

CAD 是英文名“Computer Aided Design”的首写字母的缩写,其中文名为“计算机辅助设计”。顾名思义,它是一种利用计算机辅助设计人员以快速、高效、高质量、低设计成本的方式,方便地完成产品设计任务的现代设计技术。

换言之, CAD 让设计人员用计算机为工具进行产品设计,提高产品设计效率,提高设计质量,降低设计成本。

机械产品 CAD 过程模型

Fig2 表示**机械产品 CAD 过程模型**，虚线以上部分为人工设计过程（见图中细实线部分），虚线以下为 CAD 方法（见粗实线部分）。

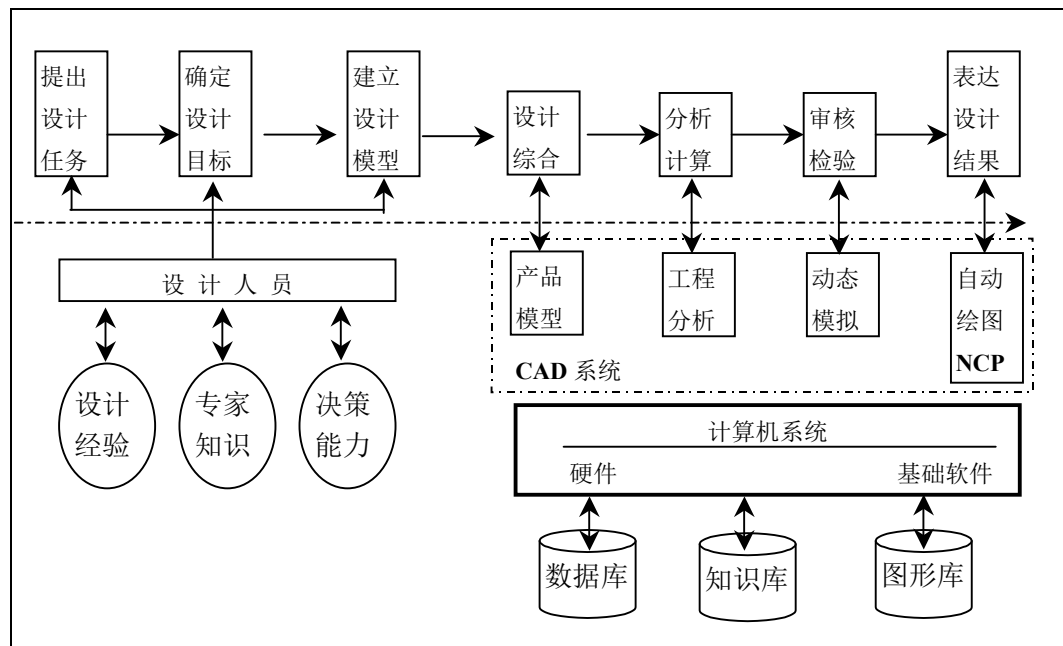


Fig2 CAD 过程模型

3. 实体造型 (Solid Modeling)

一般说来，“模型”是一个人为构造的物体，以便于人们易于观察现实世界中的物体。

实体模型 (solid Model)

实体模型是三维物体的数字化模型。它用来回答工程应用中关于三维物体所提出的一些典型的问题：诸如物体看起来象什么样子等几何性质，物体的重量、表面积、惯性矩等容积性质，以及连通性、包含关系等拓扑性质问题。

实体造型

概念：

实体造型是研究在计算机内有效表示、构造、处理工程设计中三维物体（形状）的一门科学。

意义：

形状设计是产品设计的关键——>实体造型是 CAD 的核心

4. 实体造型的产生与发展

主要是为了解决“设计与制造的分离”问题。

a. 工程图纸与手工绘制工程图（60 年代以前）

通过二维图形几何，借助于约定的符号来表示尺寸、公差、表面特性、材料等，就能为有经验的读者表达真实的三维信息。

优点：

信息量大、通用。

缺点：

- （1） 设计一套图纸周期长，修改、复制图纸更困难
- （2） 三视图表达复杂的三维形状困难
- （3） 设计的正确性不能自动保证
- （4） 不直观、不便管理

b. 二维模型与计算机绘图（60 年代）

20 世纪 60 年代出现的 CAD 系统允许设计人员用计算机而不是用图板进行二维绘图。一般称为 CA-Drawing/Drafting 系统，如 Dassault Systems 的 CADAM 软件；AutoDesk 的 AUTOCAD 软件

优缺点：解决了 a 中问题（1），但无法解决（2）（3）（4）。

随后出现了三维 CAD 系统。

c. 曲面造型系统（70 年代）

进入 20 世界 70 年代，飞机和汽车工业中遇到了大量的自由曲面问题，随着法国人 Bezier 提出的 Bezier 算法，使人们用计算机处理曲线和曲面问题变得可行，同时也使得法国 Dassault 飞机制造公司的开发者们，能在二维绘图系统 CADAM 的基础上，开发出以表面模型为特点的自由曲面建模方法，推出了三维曲面造型系统 CATIA。它的出现，标志着 CAD 技术从单纯模仿工程图样的三视图模式中解放出来，首次实现了以计算机完整描述产品零件的主要信息，同时也使得 CAD 技术的开发有了现实的基础。曲面造型系统 CATIA 为人类带来了第一次 CAD 技术革命，改变了以往只能借助油泥模型来近似表达曲面的落后的工作的方式。

（第一次 CAD 技术突破：Bezier 算法应用，3D 曲面造型系统）

曲面造型：研究复杂曲面的计算机表示与处理方法，面向曲面而不是物体。

d. 三维模型与三维实体造型（70 年代）

有了表面模型，CAM 的问题可基本解决。但由于表面模型技术只能表达形体的表面信息，难以准确表达零件的其它特性，如质量、重心、惯性矩等，对 CAE 十分不利，最大的问题在于分析的前处理特别困难。

三维实体模型：在计算机内提供对产品的完整几何定义，支持 CAD/CAM/CAE 的诸多方面。

三维实体造型：使设计人员直接在三维进行设计、修改，从而直观、高效

最初的学术系统：

TIPS （日本北海道大学、基于半空间的 CSG）

PADL-I、PADL-II（美国罗彻斯特大学、基于有界体素的 CSG）

BUILD（英国剑桥大学、B-reps）

商品化系统：

I-DEAS、CATIA、MASTER、UGII、CV 等

实体造型技术的普及应用标志着 CAD 发展史上的第二次技术革命。

e. 基于参数化、特征的实体造型（80 年代）

参数化实体造型方法。

该方法的特点如下：基于特征、全尺寸约束、全数据相关、尺寸驱动设计修改。

（实体模型中含约束与特征：表达设计意图、提高系统智能化、集成化程度）

参数化技术的应用主导了 CAD 发展史上的第三次技术革命。

决定软件先进性及生命力的主要因素是软件基础技术，而并非特定的应用技术。

f. 变量化技术（90 年代）

变量化技术既保持了参数化技术的原有优点，同时又克服了它的不足之处。它的成功应用为 CAD 技术的发展提供了更大的空间和机遇。无疑，变量化技术成就了 SDRC，也

（变量化技术）驱动了 CAD 发展的第四次技术革命。

g. 基于虚拟现实与因特网的实体造型（90 年代）

背景：DFx、CIMS、敏捷制造（Agile Manufacturing）

通过 VR 技术提高交互性、提供沉浸感

通过网络实现协同设计（team work）、异地设计、并行工程

5. 主要研究内容

(1) 实体模型及其表示方法

完备的几何模型+易于在计算机中表示与处理

完备：能提供三维物体完整的几何定义

模型+算法——>解决所有几何问题

具体表示：B-reps、CSG、分解模型、参数化模型、特征模型

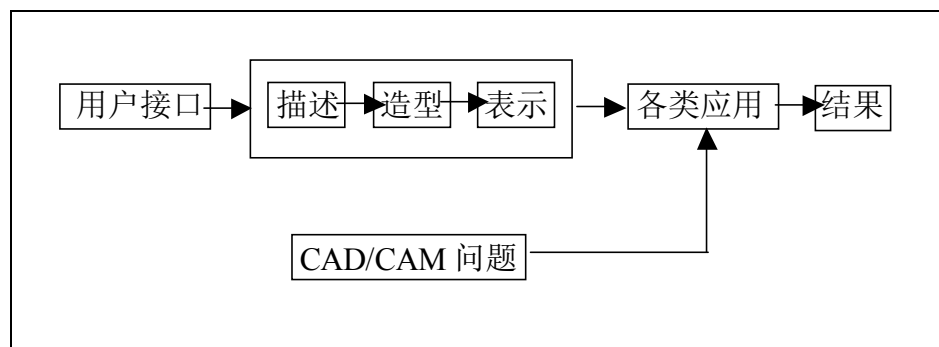
(2) 实体造型方法

构造具体物体的实体模型的方法。主要包括：

基本体素生成操作、拼合操作、局部操作、变动设计、特征编辑等。

关键问题：有效性（不自交等）、直观易用、几何覆盖域宽、效率高。

(3) 实体造型系统



系统要求：可靠性好、用户界面友好、功能强、几何覆盖域宽、效率高。

6. 课程基础

数据结构、C/C++语言、计算机图形学、点集拓扑与代数拓扑的概念

7. 课程目标

让学生了解、掌握实体造型的概念、方法，能够从事实体造型和 CAD 的研究和开发工作。

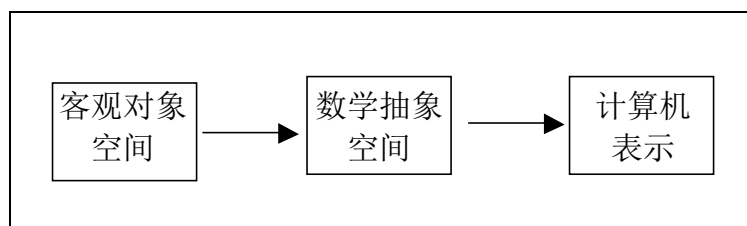
8. 主要参考文献

- [1] M. Mantyla, "An Introduction to Solid Modeling".
- [2] 唐荣锡, "CAD/CAM 技术", 北航出版社。
- [3] Jami Shah, "Parametric and feature-based CAD/CAM technology"
- [4] 唐荣锡, 汪嘉业, 彭群生, "计算机图形学教程" (第十章), 科学出版社。
- [5] 孙家广等, "计算机辅助几何造型技术", 清华大学出版社。

第二节 实体模型

1. 计算机模型的产生过程

造型的三级视图



- (1) 实际物体：根据模型的含义，我们的目标是讨论三维世界中的实际事物。
- (2) 数学对象：为了在计算机上进行造型，我们必须对感兴趣的三维实际物体进行适当的理想化。
- (3) 计算机表示：在严格地建立了我们感兴趣的数学物体后，造型工作的最后一步是对数学物体赋予适合于计算机操作的表示。

按照三级视图，我们不试图去对物体实际的物理特性造型。而代之以用如下方式表示的一些理想化的物体，即通过分析它们理想化后的特性，希望由此而得到的结果使我们的确能够获得隐藏在其背后的实际物体的信息。

举例：

线框表示

2. 体的点集模型

一个三维物体最一般的数学抽象是 E^3 的一个子集。

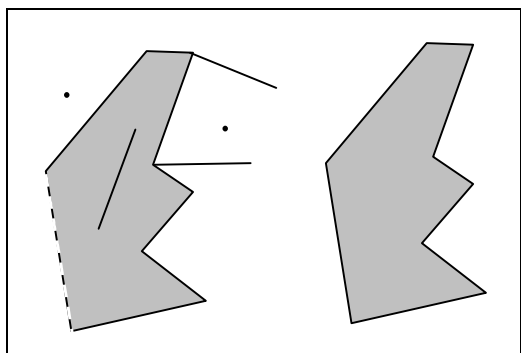
然而，是否任一三维空间的点集都对应一个三维物体呢？答案是否定的。

实际物体理想化的点集模型的优点是：我们可以使用“点集拓扑”来严格表征三维物体所希望的特性。

定义 1：设 A 为三维欧式空间中的点集，记 $r(A) = C(i(A))$ ，如果 $r(A) \equiv A$ ，则称点集 A 为正规集。

$i(A)$ ：内点集合

$C(A)$ ：闭包（内点 + 内点的极限点）



Note:

- (1) 并非所有三维空间的点集均为正规集，例如，线、面， $r(A)=\phi$ (因其内点为空)，但 $A \neq \phi$
- (2) 三维欧式空间 E^3 是正规集，但并非我们所要考虑的，现实生活中的物体。
- (3) $r(A)$ 本身一定是正规集， $r(A)$ 是 A 中最大正规子集。
- (4) 两个正规集的集合运算结果不一定是正规集。

为使集合运算有效，定义正则运算。

正则运算： $A \cup * B = r(A \cup B) = C(I(A \cup B))$ 以保证运算的有效性。

同样，对于交、差、补运算，也定义相应的正则运算。

- (5) 正规集可以是分离体的并，不一定连通。

定义 2：三维物体是 E^3 中有界、正规的点集。

推论：

- 1) 一个物体只能占有有限空间，它是封闭的，具有一定的体积。
- 2) 物体中不允许出现孤立点、悬挂的线或面。

面向工程应用的两个约定：

- 1) 物体具有刚性：即在平移、旋转变换下形状保持不变。（**刚性**）
- 2) 物体的表面是半解析的。（**表示的有限性**）

Semialgebraic: A set is semialgebraic if it can be expressed as a finite Boolean combination (via the set operators $\cup, \cap, -, c$) of sets of the form $\{(x, y, z): F_i(x, y, z) \leq 0\}$, where the F_i are algebraic.

Semianalytic: Semianalytic set are defined similarly, but the F_i are required to be analytic rather than algebraic.

3. 体的表面模型

对于工程应用中的三维物体，人们感兴趣的不是物体的内部（一般来说）而是物体的边界。当人们画一个物体时，画的是物体的边界；判断两物体是否相交时，总是判别它们

的边界是否相交；人们所能观察到的也是物体的边界；加工总是沿着零件的边界表面加工；形状由边界曲面的形状所反映；因此，需要通过三维物体的边界曲面对三维物体进行刻划、分类、表示的模型，称之为体的表面模型。

直观上看, 三维实体的边界由一系列的曲面粘合在一起形成一个包围实体的完整且封闭的蒙皮 (skin)。

3.1 二维流形

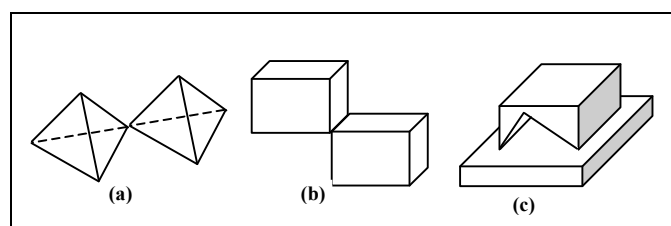
直观上看，一个曲面可认为是 E^3 的一个子集，它的本质是“二维的”：表面上的每一点（除了“开”曲面片边界线上的点外）都由属于表面上的点的“二维”区域包围。

曲面的内在的二维属性意味着我们可以通过二维模型来研究其性质。首先，我们将定义一个关于“曲面”的更抽象（且更精确）的概念，然后将按照定义于二维欧氏空间 E^2 上的一种特殊拓扑，为曲面构建一个简单的二维模型。

定义 3：设 M 为 E^2 中的一个拓扑空间，如果 M 中的每一点都存在一个邻域与 E^2 中的开圆盘同胚，则称 A 为二维流形 (2-manifold)。

Note:

- (1) 二维流形是封闭的。
- (2) 若一个二维流形和一个 r 集 A 的边界 $b(A)$ 同胚，则称 A 是 M 在 E^3 中的一个实现 (realization)。
- (3) 并非所有的 r 集都是二维流形的实现。换言之， r 集的边界曲面不一定是二维流形。



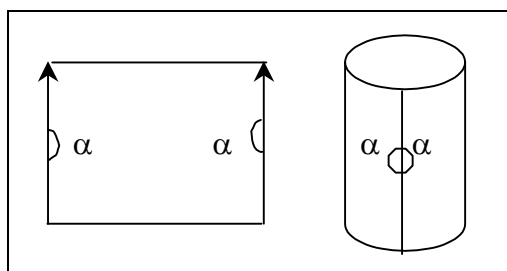
- (4) 边界为二维流形的点集不一定对应三维物体，如 Klein 瓶。

3.2 二维流形的平面图模型

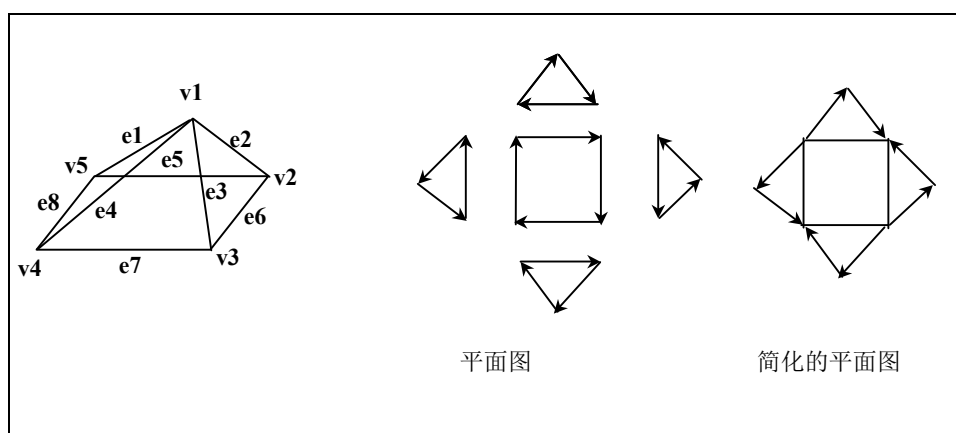
二维流形的平面图模型：由数学模型向计算机模型的转化

定义 4：平面图模型是一个有向图，这个图带有有限个顶点 $N = \{n_1, n_2, \dots, n_i\}$ ，有限条边 $A = \{a_1, a_2, \dots, a_j\}$ ，以及有限个有 N 、 A 中的点边围成的有一定走向的多边形 $R = \{r_1, r_2, \dots, r_k\}$ ，其中边、点均带有标识。

举例 1：下图(a)表示了一个四边形，它的左右边被标记为 α ；而(b)中表示的曲面（圆柱面）是将（a）中标记为 α 的两条边粘合在一起后得到的。



举例 2：金字塔的平面模型



二维流形对应一平面图，反之不然。

定理 1：一平面图为可实现的流形体平面图的充要条件为：

- (1) 对每一条边有且仅有另一条边与其对等。
- (2) 对每一个顶点，图中与该顶点相连的多边形只形成一个循环。
- (3) 平面图是可定向的，即：平面图中多边形的定向能够保证每一对对等边在它们各自所在的多边形中走向相反。(Möbius' rule)

存在没有相应的 E^3 中物理副本的二维流形，即在三维空间中无法构建该物体。(或说二维流形不能成为任何 r 集的边界。) 例如, **Klein bottle**.

4. 表示模式 (representation scheme)

设 M 表示物体的数学造型空间，在本节，我们让“solid”代表 M 中的一个元素 (entity)。

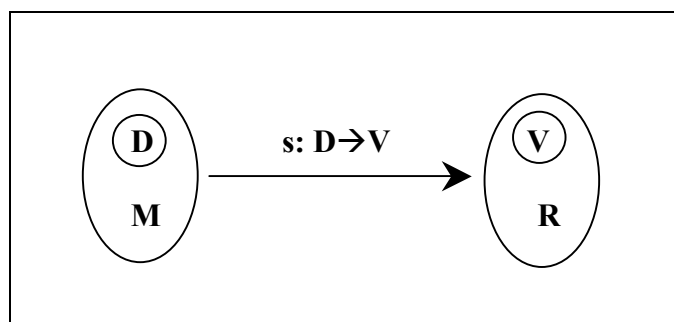
(计算机表示：由有限个标示三维实体的字符序列构成。)

定义 5 一个实体表示(solid representation)是一个有限的符号集合，它代表 M 中的一个实体 (solid)。

定义 6 一个给定的实体造型系统的表示方法定义了该造型系统的表示空间 R 。称那些能够按其句法规则 (syntax rules) 用实体造型系统实际构造的表示为可接受的 (admissible)。在本节，我们称 R 的元素为“表示”。

定义 7 一个表示模式 (representation scheme) 是一种关系 $s: M \rightarrow R$ 。 s 的定义域 (即那些在 s 下有映象 (image) 的 $m \in M$) 记为 D ，而 s 下 D 的映象记为 V 。 s 的逆映射记为 s^{-1} 。我们用 $\{m, r\} \in s$ 来表示 $r \in R$ 在 s 下 $m \in M$ 的映象。

定义 8 属于 V 的任何表示 ($r \in V$) 均被称为是有效的 (valid)。



M : 平面图模型

D : 流形体平面图模型

R : 任一计算机表示

V : 三维实体有效的计算机表示

定义 9: (不二义性) 如果任何有效的计算机表示模型有且仅有一个三维实体在 s 映射下与其对应，则称 s 为不二义的 (unambiguous), 或说信息完备的 (informationally complete) 表示模式。

定义 10 (唯一性) 如果所有三维实体 $m \in D$ 在 s 映射下有且仅有一个计算机表示与其对应，则称 s 具有唯一性。

第三节 边界表示法及其数据结构 (B-reps, Boundary Representation)

1. 边界表示法

即在计算机中通过三维物体边界的拓扑结构与几何信息来表示三维物体。
Topology 是骨架，而 Geometry 是血肉。

(1) 拓扑结构

由三维物体包含的拓扑元素以及拓扑元素间的关系组成。

拓扑元素：

主要的拓扑元素种类：

体、面、环、边、点（面是关键，但如何表示有限面？必须由边、点）

其他拓扑元素有：lump, shell 等。

拓扑关系：拓扑元素之间的连接关系。

在实体造型中，基本的拓扑元素有三种：面、边、点。它们有九种可能的连接关系。

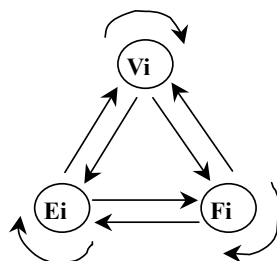


Fig. 拓扑元素的连接关系

以立方体为例，其点、边和面的连接关系如下图。

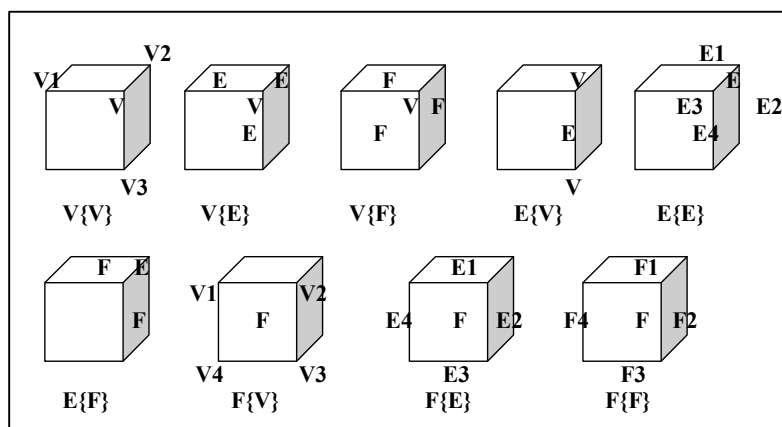


Fig. 立方体点、边、面的连接关系

Note: 九种拓扑关系之间不是完全独立的，即不必要全部显式表示。

问题：需要表示那些拓扑元素和那些拓扑关系？

主要取决于三个因素：功能需求、效率、存储量

例：线框模型：{E}、{V} E{V}

实体模型：{O}、{F}、{L}、{E}、{V}

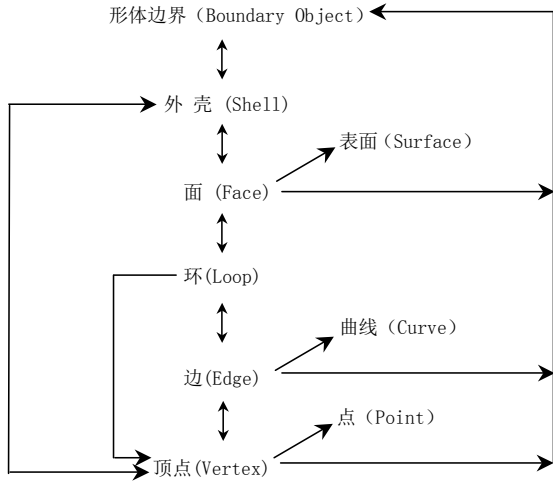
F{E}、E{V}、E{E}、E{F}、V{E}、F{V}、V{F}

(2) 几何信息

用来表示上述元素的几何性质和度量关系，一般描述物体的大小、尺寸、位置的信息。具体的边界几何信息包括：面的方程、边的方程、点的坐标。

在具体的数据结构中，拓扑元素与它的几何信息一一对应：

- 顶点: Vertex → Point 含点的坐标
- 边: Edge → Curve 曲线方程（控制顶点）
- 面: Face → Surface 曲面方程（控制顶点）



2. 翼边 (wing-edge) 数据结构

翼边结构于 70 年代由 Standford 大学的 B. G. Baumgart 作为多面体的边界表示法提出，最早用于从图象重建三维物体中。

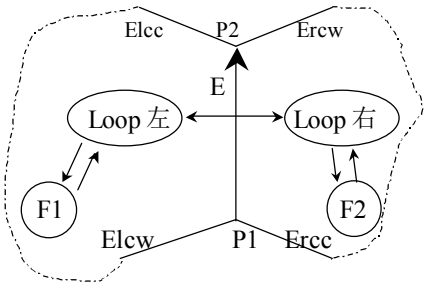
拓扑元素种类: {F}、{E}、{L}、{V}

拓扑关系: $E\{V\}$ 、 $E\{E\}$ 、 $E\{F\}$

具体的数据结构:

边的数据结构:

E:	P1	Loop 左	Ercc Erclw
	P2	Loop 右	Elcc Elcw



P1, P2: 为边的两个端点（起始端点和终止端点） $E\{V\}$

Elcc, Elcw, Erclw, Ercc 为与边 E 相连的 4 条翼边 $E\{E\}$

Elcc: 左环中 E 的沿逆时针方向的下一条边

Elcw: 左 顺

Ercc: 右 逆

Ercw: 右 顺
 Loop 左, Loop 右: 为与边 E 相连且位于其左、右侧的环。E {F}

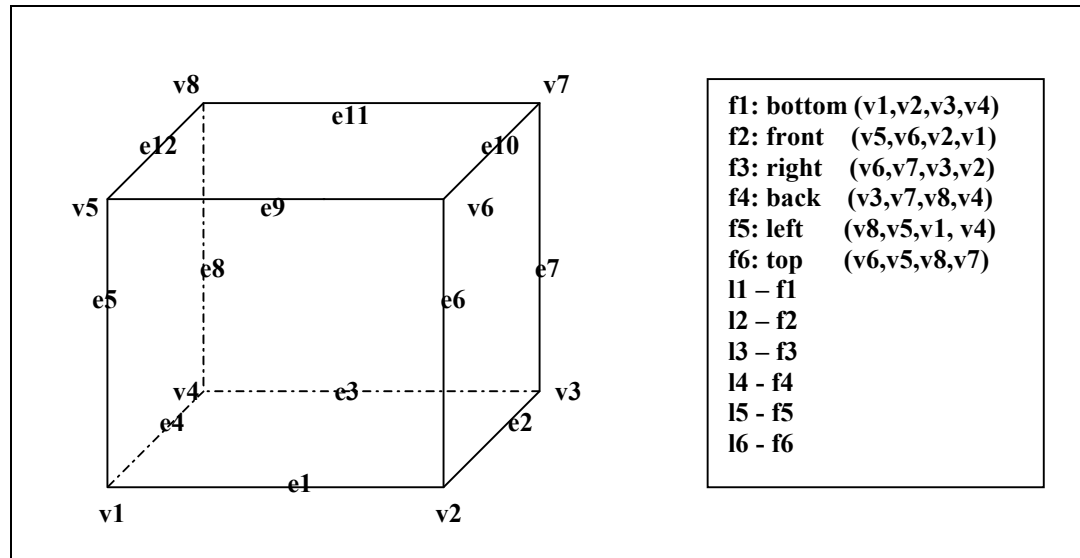
其它拓扑元素的数据结构:

面: 为一链表, 主要包含: 首环指针、体指针、曲面指针等

环: 为一链表, 主要包含: 首边指针、面指针、曲线指针等

点: 含一条与其相连的边指针, 几何点指针等。

例:



边	P1	P2	L左	L右	Elcc	Elcw	Ercc	Ercw		顶点	首边	坐标		环	首边
e1	v1	v2	12	11	e6	e5	e4	e2		v1	e1	...		l1	e1
e2	v2	v3	13	11	e7	e6	e1	e3		v2	e2	...		12	e9
e3	v3	v4	14	11	e8	e7	e2	e4		v3	e3	...		13	e6
e4	v4	v1	15	11	e5	e8	e3	e1		v4	e4	...		14	e7
e5	v1	v5	15	12	e12	e4	e1	e9		v5	e9	...		15	e12
e6	v2	v6	12	13	e9	e1	e2	e10		v6	e10	...		16	e9
e7	v3	v7	13	14	e10	e2	e3	e11		v7	e11	...			
e8	v4	v8	14	15	e11	e3	e4	e12		v8	e12	...			
e9	v5	v6	16	12	e10	e12	e5	e6							
e10	v6	v7	16	13	e11	e9	e6	e7							
e11	v7	v8	16	14	e12	e10	e7	e8							
e12	v8	v5	16	15	e9	e11	e8	e5							

分析:

- (1) 数据结构以边为中心, 用边来联系顶点、邻边以及环、面、体, 从而能够方便地导出其它未显式表示的拓扑关系

A. 从 Loop 的一条边能方便地找到 Loop 的所有边 → F {E}

All_edges (1p)

```

{
    te = e = lp->e0;
    if(e->loop左== lp) {
        do {
            if(te->loop左==lp) te =te->Elcc;
            else te=te->Ercc;
        } while(te != e);
    }
    else {
        do {
            if(te->loop左==lp) te = te->Elcw;
            else te=te->Ercw;
        } while (te != e)
    }
}
}

```

B. 从顶点的一条边能方便地找到与该点相连的所有边。

```

te = e = v->e0;
do {
    if(te->p1==v)
        te = te->Elcw;
    else te = te->Ercw;
} while(te != e);

```

- (2) 保证每条边仅与两张面相连。
- (3) 由于边是有向的,从而可以通过首边前进的方向隐含确定物体表面的外法向。
- (4) wing-edge 的问题:
 - 边的方向可能与面的走向不一致。
 - 构造较麻烦。

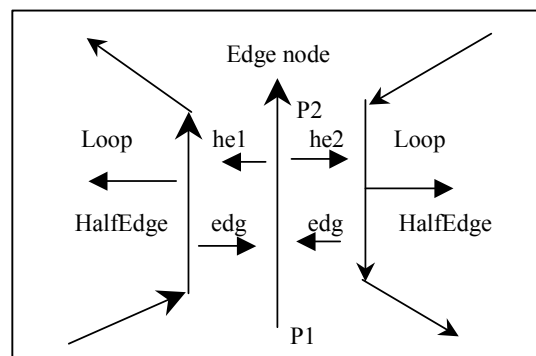
3. 半边 (half-edge) 数据结构

半边数据结构是翼边数据结构的变形,主要出发点是要改进翼边数据结构,使其在边的数据结构的构造和使用方面更加高效。已成为主流边界表示数据结构。Mantyla 的半边数据结构广为引用。

3.1 核心思想

在数据结构中将一条物理边当作两条边表示,其中每一条边只与一个面相关,称为半边。所谓半边,本质上讲只表示物理边一半的信息。

半边、边和顶点构成整个数据结构的最核心部分。



3.2 半边数据结构的层次结构

为表示图 $\{N, A, R\}$ ，我们将使用一个五层结构，该结构包含了 Solid, Face, Loop, HalfEdge 和 Vertex。

3.3 C 语言描述

```
typedef float      vector[4];
typedef float      matrix[4][4];
typedef short      Id;
typedef struct solid Solid;
typedef struct face  Face;
typedef struct loop  Loop;
typedef struct halfedge HalfEdge;
typedef struct vertex Vertex;
typedef struct edge  Edge;
typedef union nodes Nodes;

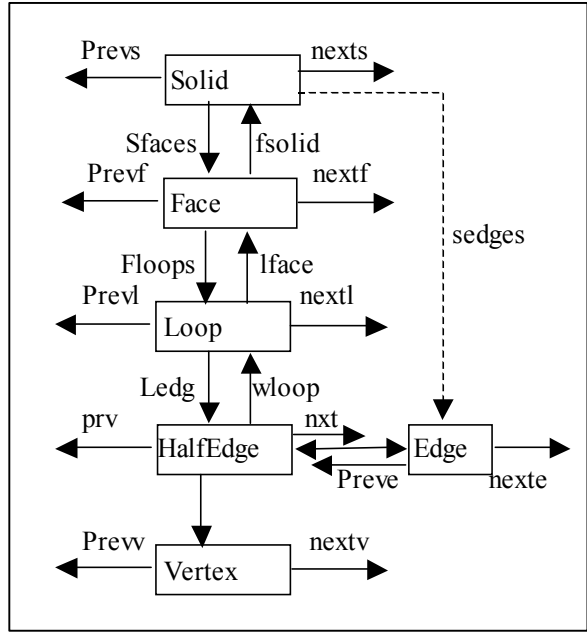
struct solid {
    Id      solidno; // solid identifier
    Face     *sfaces; // pointer to list of faces
    Edge     *sedges; // pointer to list of edges
    Vertex   *sverts; // pointer to list of vertices
    Solid     *nexts; // pointer to next solid
    Solid     *prevs; // pointer to previous solid
};

struct face {
    Id      faceno; // face identifier
    Solid    *fsolid; // back pointer to solid
    Loop     *flout; // pointer to outer loop;
    Loop     *floops; // pointer to list of loops
    Vector   feq; // face equation
    Face     *nextf; // pointer to next face
    Face     *prevf; // pointer to previous face
};

struct loop {
    HalfEdge *lledge; // ptr to ring of halfedges
    Face     *lface; // back pointer to face
    Loop     *nextl; // pointer to next loop
    Loop     *prevl; // pointer to previous loop
};

struct edge {
    HalfEdge *he1; // pointer to right halfedge
    HalfEdge *he2; // pointer to left halfedge
    Edge     *nexte; // pointer to next edge
    Edge     *preve; // pointer to previous edge
};

struct halfedge {
```



```
Edge     *edg; // pointer to parent edge
Vertex   *vtx; // pointer to starting vertex
Loop     *wloop; // back pointer to loop
HalfEdge *nxt; // pointer to next halfedge
HalfEdge *prv; // pointer to previous halfedge
};

struct vertex {
    Id      vertexno; // vertex identifier
    HalfEdge *vedge; // pointer to a halfedge
    vector   vcoord; // vertex coordinates
    Vertex   *nextv; // pointer to next vertex
    Vertex   *prevv; // pointer to previous vertex
};

union nodes
{
    Solid    s;
    Face     f;
    Loop     l;
    HalfEdge h;
    Vertex   v;
    Edge     e;
};
```

3.4 存取方法

层次式存取 (Hierarchical Access)

许多例程（如产生图形输出、坐标变换等）仅需要简单地扫描整个数据结构，并在每个结点处进行某种运算。借助于半边数据结构的层次结构，这很容易做到。

```
void    listsolid(s)
Solid   *s;
{
    Face      *f;
    Loop      *l;
    HalfEdge   *he;

    f = s->sfaces;
    while(f)
    {
        printf("face %d:\n", f->faceno);
        l = f->floops;
        while(l)
        {
            printf("loop:");
            he = l->ledg;
            do
            {
                printf(" %d: (%f %f %f)",
                    he->vtx->vertexno, he->vtx->vcoord[0],
                    he->vtx->vcoord[1], he->vtx->vcoord[2]);
            }
            while((he = he->nxt) != l->ledg);
            printf("\n");
            l = l->nextl;
        }
        f = f->nextf;
    }
}
```

Prog. Hierarchic access (列出每个面的顶点)

跨边存取 (Access Over Edges)

边结点是数据结构导航的关键。特别地，从一个不表示空环 (empty loop) 的半边出发，可以很方便地存取与它匹配的半边。

```
#define mate(he) (((he) == (he)->edg->he1) ? (he)->edg->he2 : (he)->edg->he1)
```

换言之，若半边 he 是边的“左”半部分，其“匹配”半边为边的“右”半部分；反之亦然。

```
void    listneighbors(v)
Vertex  *v;
{
    HalfEdge   *adj;

    adj = v->vedge;
    if(adj)
        do
```



```

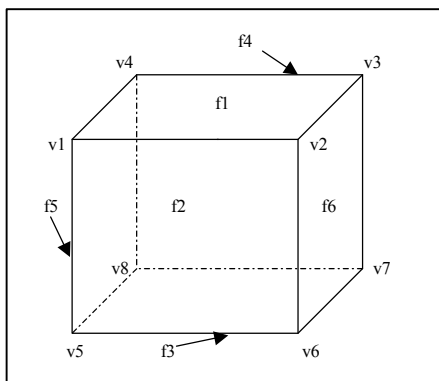
    {
        printf("%d ", adj->nxt->vtx->vertexno);
    }
    while((adj = mate(adj)->nxt) != v->vedge);
    else    printf("no adjacent vertices");
    printf("\n");
}

```

3.5 分析

- (1) 边结构包含了翼边结构的所有信息，内存相当。
- (2) 半边结构使得每一面的边界数据与其他面相对独立，从而能够提高搜索所有边的效率，提高建立数据结构的效率。
- (3) 由于在数据结构中边的数目是真正边的数目的两倍，造成某些算法更耗时，如画线框、求交等。
解决办法：在数据结构中增加物理边表。

4. 基于面的边界数据结构



最简单的情形：

面用多边形表示，每一多边形由一串坐标三元组组成，实体是一组面构成。

f1:

((x1, y1, z1), (x2, y2, z2), (x3, y3, z3), (x4, y4, z4))

f2:

...

这种表示常用于图形系统的元文件 (metafile) 中。

FAG (Face Adjacency Graph)

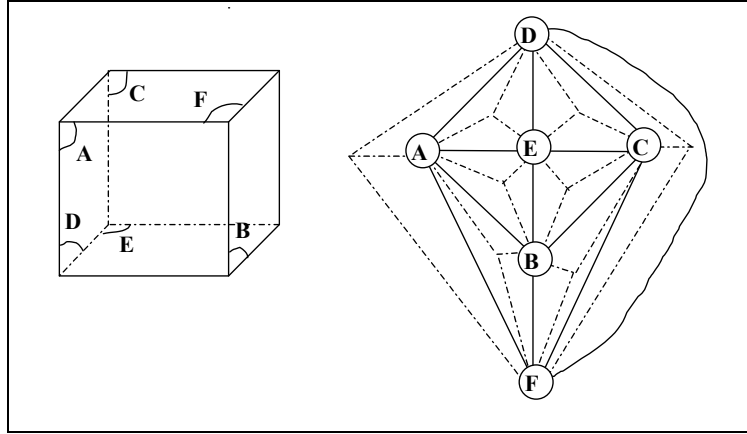
特点：

- (1) FAG 表示选择面作为物体的定义元素，边一面关系作为物体元素间的基本关系。
- (2) SFAG (Structured Face Adjacency Graph) 用于反映物体造型过程，该层次结构提供了不同层次细节下物体的表示。

定义：一个物体 S 的面邻接图是一个满足下述条件的超图 $G = (N, A, H)$ ：

- (i) 对于 F_s 中的每一个面 f ， N 中存在唯一一个结点 n 与 f 对应；
- (ii) 对于 E_s 中的每一条边 e (边 e 由 S 中的两个面 f_1 和 f_2 共享)， A 中存在唯一一条弧，连接 N 中的两个结点 n_1 和 n_2 (n_1 、 n_2 分别与 f_1 、 f_2 对应)；
- (iii) 对于 V_s 中的每一个顶点 v ，定义 S 中所有相交于点 v 的面集为 $F_v (\subset F_s)$ ，定义与 F_v 中的面相应的 G 中结点子集为 $N_v (\subset N)$ ；则对于 V_s 中的每一顶点 v ，存在唯一一个 H (称为超弧) 中的元素，连接 N_v 中的结点。

举例：立方体及其 FAG 表示（在 FAG 表示中，实线表示弧，而虚线表示超弧）



SFAG: FAG 结构表示

在任何一种物体表示方法中，通常将**物体的高层描述**与**底层细节描述**（或说**形状属性**）区分开是重要的。

形状属性：相对于整个物体的大小而言，具有一个小的尺寸。如凸台（protrusion）、凹陷（depression）、柄（handle）、通孔（through hole）等。——— 特征

将形状属性视为包围它的物体的面集。因此，

一个物体的边界表示 $B = (V, E, F)$ 可视为由以下两部分组成：

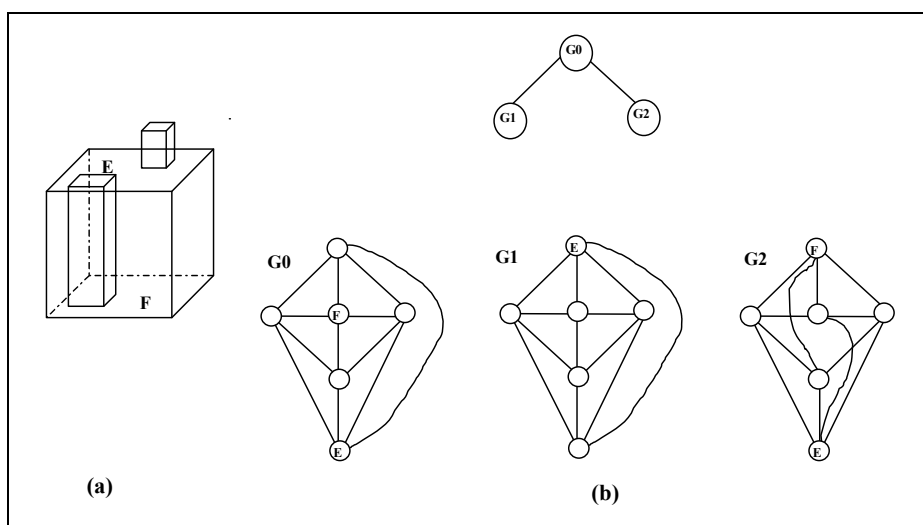
- (1) 在最高抽象层的物体 S 的表示，记为 $\bar{B} = (\bar{V}, \bar{E}, \bar{F})$ ；
- (2) S 的属性的边界描述，每一个记为 $B^{(r)} = (V^{(r)}, E^{(r)}, F^{(r)})$, $r = 1, 2, \dots, k$ 。且每一属性又可表示为一个主表示加上它的底层属性，如此生成在更精细层次下的 S 的边界表示。

SFAG 定义：

SFAG 是一个二元组 $\gamma = (G, T)$ ，其中 T 是描述 γ 的结构树； G 是一簇 FAG， G 的每一组件 G_i 与 T 中唯一一个结点 t_i 相关。

与 T 的根结点相关的组件 G 是表示 $\bar{B}$ 的 FAG；而其它组件 G_i 是一个形状属性的 FAG 表示。

- (1) 由于 S 的任一形状属性均与其一个或多个元素（面、边、顶点）相关，因此，每一个 G_i （除根结点外），均与图中的一个元素（结点、弧、超弧）或与图 G_j 的子图相关（ G_j 是与 T 中 t_i 的父结点 t_j 对应的图）。
- (2) G_i 与扩张成 G_i 的 G_j 元素或子图的关系定义为一个结点 (node) 集，该集合中的结点同属于 G_i 和 G_j ，称为 G_i 的边界结点 (boundary nodes)。图 G_i 的边界结点集记为 BN_i 。



物体 S 的完整的 FAG 可从其 SFAG 获得。

方法：递归地应用 refinement transformation 到其 macroelements 和 macrosubgraphs 上。

反之，物体的 SFAG 也可从其 FAG 得到。

优点：以面为中心组织体的数据结构，易于描述形状特征间的空间位置关系，可以有效支持特征识别。

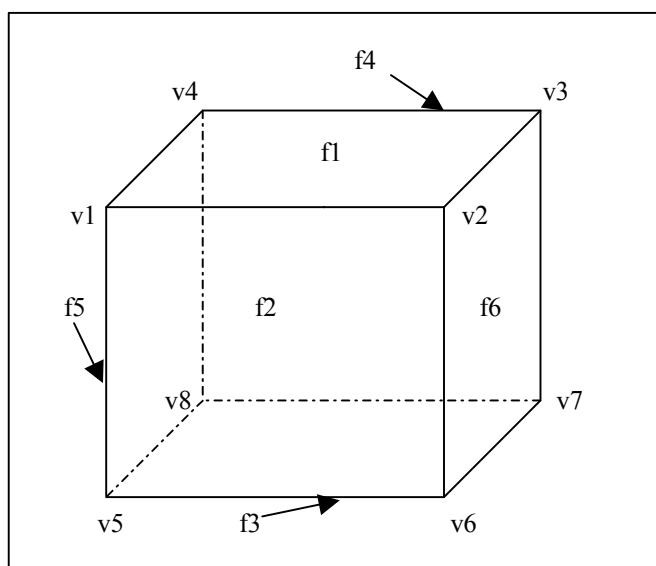
5. 基于点的边界数据结构

拓扑元素：

{V}：坐标

{F}：面所包含的所有点。依据一定的走向顺序排列。

拓扑关系：F {V}



Example:

f1: v1, v2, v3, v4
 f2: v1, v5, v6, v2
 f3: v5, v8, v7, v6
 f4: v3, v7, v8, v4
 f5: v1, v4, v8, v5
 f6: v2, v6, v7, v3
 v1: (x1, y1, z1)
 v2: (x2, y2, z2)
 ...
 v8: (x8, y8, z8)

Note: (1) 对于多面体是完备的。而对曲面物体则不完备，因为曲面不可能仅用两端点唯一表示。

(2) 面中顶点排列一致（右手法则），对于许多算法有用，如隐线消除或隐面消除。

第四节 基于边界模型的造型方法

造型操作的要求：

- (1) 符合设计者的设计习惯
- (2) 直观
- (3) 易用
- (4) 具有一般性并且几何覆盖域宽
- (5) 有效性
- (6) 高效率

造型操作是具有等级的、是具有层次的。

直接为用户所使用，即高级操作；不直接为用户所使用的操作为低级操作。

核心操作：实体模型之间的变换；非核心操作：显示、物性计算等操作。

一般操作：边界表示数据结构的生成操作

1. 欧拉操作

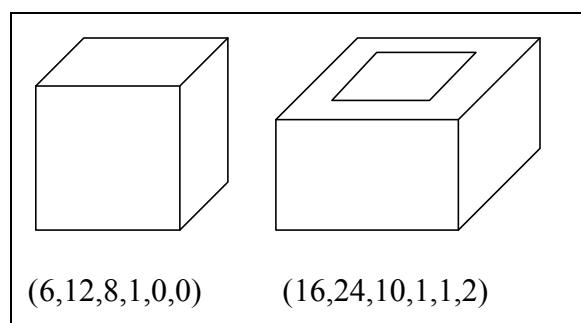
由 B.G. Baumgart 于 1972 年提出欧拉操作的概念，目的是提供有效、正确地建立三维物体复杂的边界数据结构的方法（保证有效性、保证通用性）。

1.1 欧拉操作的设计思想

仅建立拓扑结构，假设几何信息已知。

欧拉公式： $v(\text{点数}) - e(\text{边数}) + f(\text{面数}) = 2(s(\text{分离体数}) - h(\text{通孔数})) + r(\text{内环数})$

举例：



- (1) 提供少数几个公用的边界数据结构生成操作，使得三维物体复杂的边界数据结构可通过这些公用操作逐步构造完成。
- (2) 基于欧拉公式保证使用公用数据结构生成操作所生成的边界数据结构具有拓扑有效性。

1.2 欧拉操作的选取

原则：取具有明显几何意义的基：造点、造边、造环、造面、造体

I. C. Braid 所取的一组基:

Operator	Base Vector					
	v	E	f	h	r	s
mev	1	1	0	0	0	0
mef	0	1	1	0	0	0
mvfs	1	0	1	0	0	1
kemr	0	-1	0	0	1	0
kfmrh	0	0	-1	1	1	0

By historical convention, the operators are usually denoted by rather cryptic, mnemonic names. The key to the names to be used in this text is given in the below:

M - make	V - vertex	H - hole
K - kill	E - edge	R - ring
S - split	F - face	
J - join	S - solid	

For instance, the name “mev” translates as “Make Edge, vertex”.

1.3 欧拉操作的功能与实现

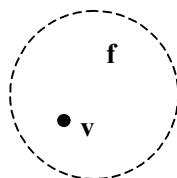
具体设计欧拉操作算法时应注意:

- (1) 通用性: 适用于所有情况;
- (2) 有效性: 除拓扑元素的数目满足欧拉公式外, 更重要的是要保证欧拉操作生成的平面图为流形平面图 (三个有效条件)。

1) mvfs(Id sn, Id f, Id v, float x, float y, float z)

功能: 定义一个体、一个面、一个外环、一个点。

图示:



算法:

```

Solid  *mvfs(fn, vn, x, y, z)
Id      fn, vn;
float   x, y, z;
{
    Solid      *newsolid;
    Face       *newface;
    Vertex     *newvertex;
    HalfEdge   *newhe;
    Loop       *newloop;

    newsolid = (Solid *) new(SOLID, NIL);
    newface = (Face *) new(FACE, newsolid);
    newloop = (Loop *) new(LOOP, newface);
    newvertex = (Vertex *) new(VERTEX, newsolid);
    newhe = (HalfEdge *) new(HALFEDGE, NIL);

    newface->faceno = fn;
    newface->flout = newloop;

```

```

newloop->ledg = newhe;
newhe->wloop = newloop;
newhe->nxt = newhe->prv = newhe;
newhe->vtx = newvertex;
newhe->edg = NIL;
newvertex->vertexno = vn;
newvertex->vcoord[0] = x;
newvertex->vcoord[1] = y;
newvertex->vcoord[2] = z;
newvertex->vcoord[3] = 1.0;

return(newsolid);
} // mvfs

```

逆算子: kvfs(), 消去体的最后一个点、一个外环、一个面及体本身。

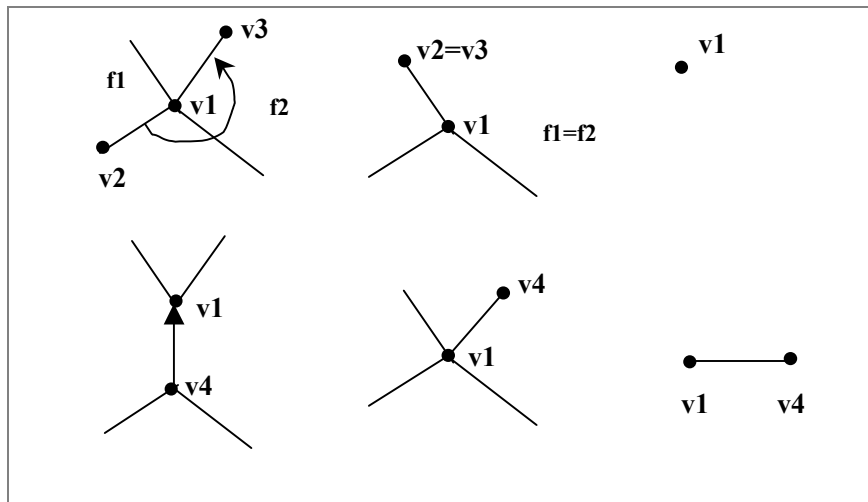
2) mev(s, f1, f2, v1, v2, v3, v4, x, y, z)

Id s, f1, f2, v1, v2, v3, v4;

Float x, y, z;

Smev(s, f1, v1, v4, x, y, z)

功能: 定义一个新点, 同时定义一条连接新点与另一给定点的边。



算法:

```

int      mev(s, fn, vn1, vn2, x, y, z)
Solid    *s;
Id        fn, vn1, vn2;
float     x, y, z;
{
    Face    *oldface;
    HalfEdge *he;

    if((oldface = fface(s, fn)) == NIL)
    {
        fprintf(stderr, "mev: face %d not found\n",
fn);
        return(ERROR);
    }
    if((he = fhel(oldface, vn1)) == NIL)
    {
        fprintf(stderr, "mev: vertex %d not found in
face %d\n",
vn1, fn);
        return(ERROR);
    }
    lmev(he, he, vn2, x, y, z);

    void      lmev(he1, he2, vn, x, y, z)
    HalfEdge *he1, *he2;
    Id        vn;
    float     x, y, z;
    {
        HalfEdge *he;
        Vertex    *newvertex;
        Edge      *newedge;

        newedge = (Edge *) new(EDGE,
he1->wloop->lface->fsolid);
        newvertex = (Vertex *) new(VERTEX,
he1->wloop->lface->fsolid);
        newvertex->vcoord[0] = x;
        newvertex->vcoord[1] = y;
        newvertex->vcoord[2] = z;
        newvertex->vcoord[3] = 1.0;
        newvertex->vertexno = vn;
        return(SUCCESS);
    }
}

```

```

he = he1;
while(he != he2)
{
    he->vtx = newvertex;
    he = mate(he)->nxt;
}

addhe(newedge, he2->vtx, he1, MINUS);
addhe(newedge, newvertex, he2, PLUS);

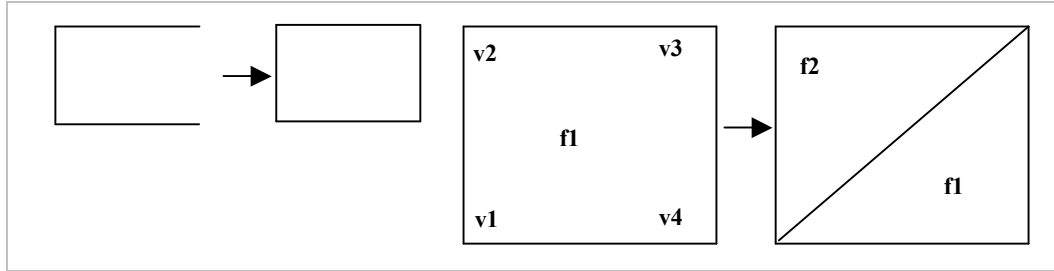
newvertex->vedge = he2->prv;
he2->vtx->vedge = he2;
}

```

逆操作: kev() 消去一个点及与其相连的一条边, 该点仅与一条边相连。

3) mef(s, f1, v1, v2, v3, v4, f2)
 Id s, f1, v1, v2, v3, v4, f2;

功能: 以两给定点为端点定义一条新的边, 同时定义一个新的面, 新的环。



算法:

```

int mef(s, vert1, vert2, fac1, fac2)
Solid *s;
Id vert1, vert2;
Id fac1, fac2;
{
    Face *oldface, *newface;
    register HalfEdge *he1, *he2;

    if(!(oldface = fface(s, fac1)))
    {
        fprintf(stderr, "mef: face %d not found\n",
fac1);
        return(ERROR);
    }

    /* get vert1 */
    if(!(he1 = fhe1(oldface, vert1)))
    {
        fprintf(stderr, "mef: vertex %d not found\n",
vert1);
        return(ERROR);
    }

    /* get the next occurrence of vert2 */
    he2 = he1;
    if(he2->vtx->vertexno != vert2)
    do
    {
        he2 = he2->nxt;
        if(he2 == he1)
        {
            fprintf(stderr, "mef: vertex %d not
found\n",
                vert2);
            return(ERROR);
        }
    }

    while(he2->vtx->vertexno != vert2);

    newface = lmef(he1, he2, fac2);

    return(SUCCESS);
}

Face *lmef(he1, he2, nf)
HalfEdge *he1, *he2;
Id nf;
{
    Face *newface;
    Loop *newloop;
    Edge *newedge;
    HalfEdge *he, *nhe1, *nhe2, *temp;

    newface = (Face *)new(FACE,
he1->wloop->lfac->fsolid);
    newloop = (Loop *)new(LOOP, newface);
    newedge = (Edge *)new(EDGE,
he1->wloop->lfac->fsolid);
    newface->faceno = nf;
    newface->flout = newloop;

    he = he1;
    while(he != he2)
    {
        he->wloop = newloop;
        he = he->nxt;
    }

    nhe1 = addhe(newedge, he2->vtx, he1, MINUS);
    nhe2 = addhe(newedge, he1->vtx, he2, PLUS);

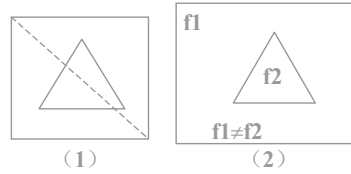
    nhe1->prv->nxt = nhe2;
    nhe2->prv->nxt = nhe1;
    temp = nhe1->prv;
}

```

```
    nhe1->prv = nhe2->prv;
    nhe2->prv = temp;

    newloop->ledg = nhe1;
    he2->wloop->ledg = nhe2;

    return(newface);
}
```

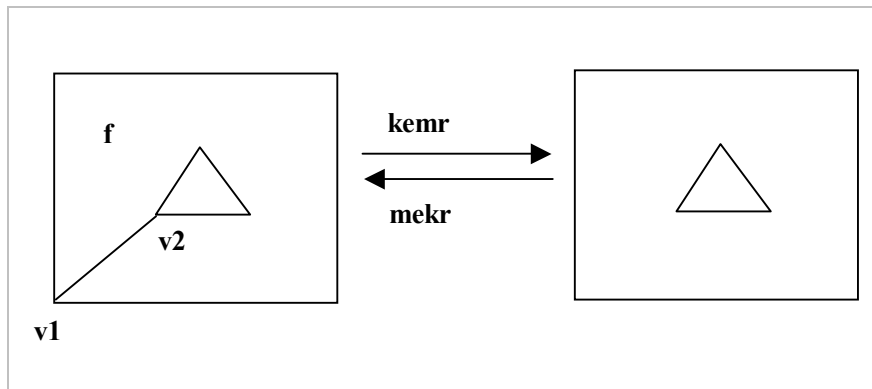



逆操作: $\text{kef}()$: 消去一个面, 一个环及面中的一条边, 这里边与两个面相连。

4) $\text{kemr}(s, f, v1, v2)$

Id $s, f, v1, v2$;

功能: 消去环中的一条边, 定义一个内环。



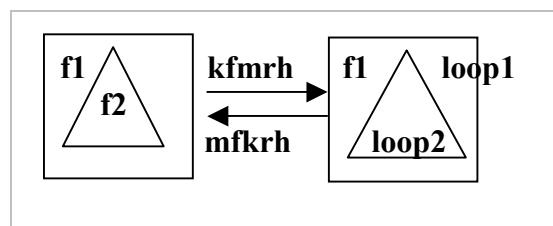
算法注意点: 检查是否消去的边仅与一面相连。

逆操作 $\text{mekr}()$: 消去一条边, 生成一个内环, 这里边的两端点共面不共环。

5) $\text{kfmrh}(\text{Id } s, \text{Id } f1, \text{Id } f2) \{ \text{kfksmr}() \}$

功能: 删除一个面, 并将其定义为另一个面的内环, 进而在体中生成一个通孔或将两物体合并成一个物体。

图示:

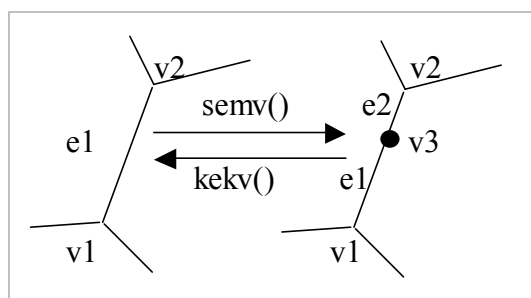


算法注意点: 判别 $f1, f2$ 是否属于同一个物体。是: $\text{kfmrh}()$; 否: $\text{kfksmr}()$ 。

逆操作: $\text{mfkrh}()$: 消除一个内环, 一个通孔, 定义一个新面。

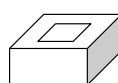
6) $\text{semv}(e, p)$

功能: 在给定边上定义一个新的点, 同时以新点和给定边的终点为两端点定义一条新的边。

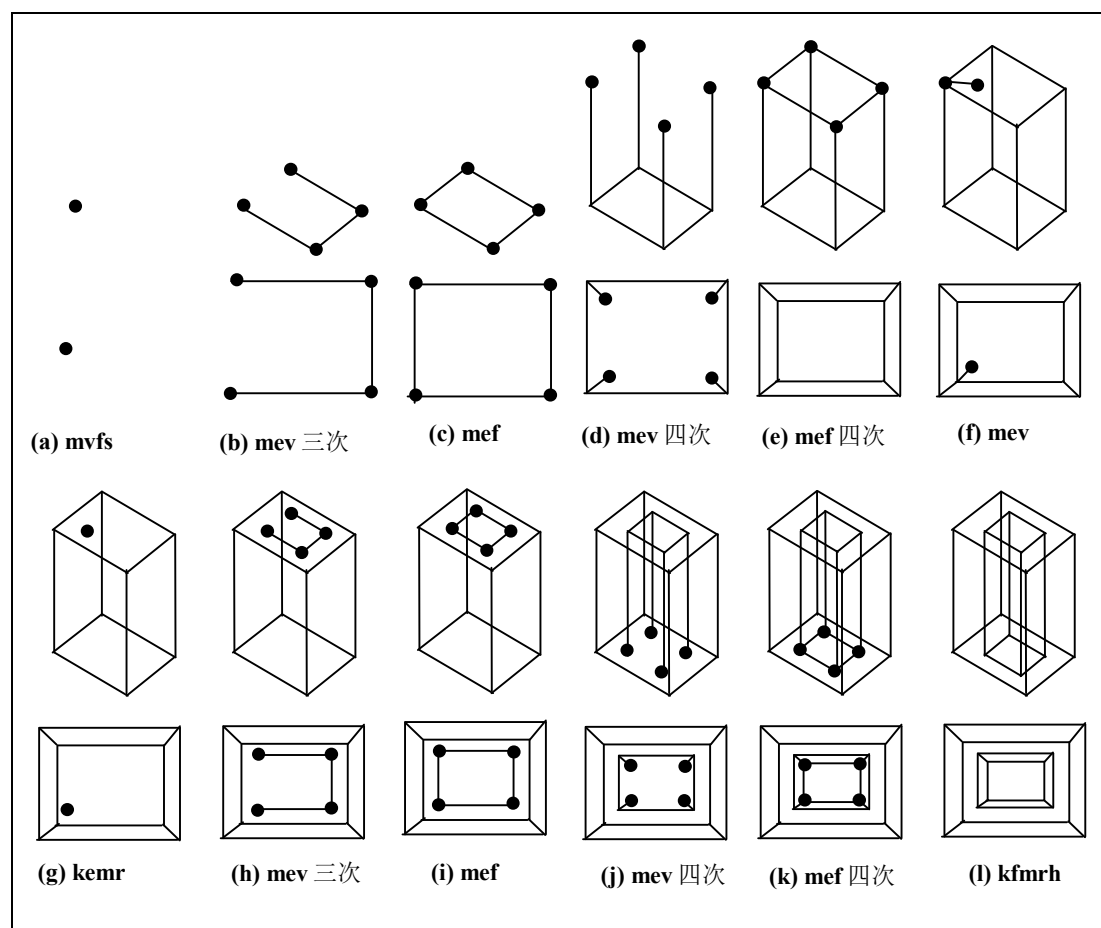


逆操作：kekv()，删除一个点，一条边，这里点与两条共线边相连。极少用。

1.4 举例



用欧拉操作构造右侧物体（长方体内带方形通孔）



1.5 有关欧拉操作的几何结论

在使用欧拉操作进行边界数据结构生成时，有两个问题需要解决：

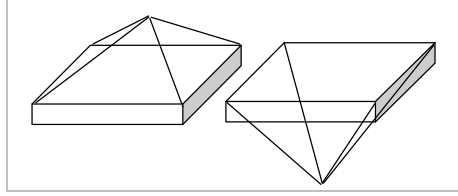
- (1) 欧拉操作的构造能力；
- (2) 所构造的边界表示的拓扑有效性

Mantyla 回答了上述问题，他从理论上得到了以下结果：

- (1) 所有流形体的边界表示都可由欧拉操作构造出来。
- (2) 由欧拉操作构造出的边界表示在拓扑结构上一定是有效的。
- (3) 将这种表示正确地嵌入欧几里德空间结果一定是流形体。

以上结果表明，欧拉操作作为正确构造流形体的边界表示提供了一种自然的方法（A. A. G. Requicha 的话）。上述第三点，实际上是几何有效性问题，即边界表示是否对应流形体，既有拓扑有效性问题，也有几何有效性问题。

举例：



几何有效性的检查方法：边界元素不自交，边界元素之间仅相交在公共边界上。

1.6 欧拉操作的代数性质

$$M = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & -1 & 1 & 1 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 \\ 1 & -1 & 1 & 2 & -1 & -2 \end{bmatrix}$$

$$M^{-1} = \frac{1}{12} \begin{bmatrix} 9 & -5 & 2 & -2 & 3 & 1 \\ -3 & 5 & -2 & 2 & -3 & -1 \\ 3 & 7 & 2 & -2 & 3 & 1 \\ -6 & 2 & 4 & 8 & -6 & 2 \\ 3 & 5 & -2 & 2 & 9 & -1 \\ 6 & -2 & 8 & 4 & -6 & -2 \end{bmatrix}$$

2. 扫成操作

扫成操作具有简单、高效、易用的特点。适用范围：二维半物体。

扫成体（sweeping 体）指由一个平面二维区域沿一条给定的轨迹线扫描出的三维空间点集，是二维半物体。构造扫成体的算法称扫成操作。通常按轨迹线的类型分成三类：

2.1 平移扫成体（Translational sweeping, extrusion）

通过将一平面区域沿某矢量方向移动一距离构造出一个柱类体。

- 轨迹线是一段直线
- 平面区域可以是由直线、圆弧、自由曲线构成的任何连同区域。

算法实现:

几何信息构造:

输入点 (老点) $\rightarrow$ 新点 (新点 = 老点 + dV) (d: 距离; V: 矢量) $\rightarrow$ 新边

输入边 (老边) $\rightarrow$ 新边 $\rightarrow$ 新面

直线 $\rightarrow$ 平面,

圆弧 $\rightarrow$ 圆柱面;

自由曲线 $\rightarrow$ 自由曲线与直线的直积曲面 (直纹面))

拓扑信息构造:

```
sweep(s: solid, f: face, v: vector, d: real)
begin
  l = loop of f;
  firstv = first vertex of l;
  firstup = firstv + dv;
  mev (firstv, firstup, newe);
  prevup = firstup; nextv = next vertex of l;
  while (nextv not equal to firstv) do
    begin
      up = nextv + dv;
      mev(nextv, up, newe);
      mef(prevup, up, f, newf, newe); // 生成新边、新面
      prevup = up;
      nextv = next vertex of l;
    end
  end
  mef(prevup, firstup, f, newf, newe);
end
```

Note:

- (1) 与输入面 F 相关的另一输入面被隐藏了。
- (2) 多环情况未作考虑。增加处理多环功能的关键是最后用一次 kfmrh()。
- (3) 需建立曲线、曲面的方程与结构。

2.2 旋转扫成 (rotational sweeping)

通过将一平面区域绕一轴线旋转一个角度, 构造出一个旋转体。

- 轨迹线为圆或圆弧
- 当平面区域退化为一折线段时, 旋转角度必须是 360 度, 才能构造三维物体。此时需补上下两底面, 即: 旋转体 = 旋转面 + 上下两底面
- 为保证几何有效性, 平面区域或折线段应完全位于轴线的一侧。

算法实现:

几何构造:

输入点 (老点) $\rightarrow$ 新点 $\rightarrow$ 新边

输入边 $\rightarrow$ 新面

直线边 (水平线 $\rightarrow$ 平面; 垂直线 $\rightarrow$ 圆柱面; 其他 $\rightarrow$ 圆台或圆锥)

圆弧 $\rightarrow$ 环面

自由曲线 $\rightarrow$ 自由曲面 (自由曲线与圆弧的直积)

拓扑构造: 与平移扫成算法基本相同。

Note: 可通过旋转机制构造复杂的旋转多面体, 是平面多面体系统中处理旋转面的必备手段。

2.3 广义扫成运算 (general sweeping)

将一平面区域沿任一空间轨迹线扫描, 构造出三维物体。在扫描中平面区域本身也可按一定的规则变化。

- 轨迹线是任意空间曲线
- 广义 sweeping 包含平移、旋转 sweeping, 造型能力很强。
- 难点: 几何构造困难, 具体是其边界曲面难以用统一、精确的数学公式描述。

(a) 理论表示法:

$$\Gamma(s, u) = \mathbf{r}(s) + C_1(u)\mathbf{n}(s) + C_2(u)\mathbf{b}(s)$$

这里 $\mathbf{r}(s)$ 为空间轨迹线, 要求曲率非零。

s 为弧长参数, $\mathbf{n}$ 为 Frenet 标架的法向量, $\mathbf{b}$ 为副法向量。

$C(u) = (C_1(u), C_2(u))$ 为参数化的平面区域边界线, 假定在扫描中 $C(u)$ 不变。

(b) 实用表示法

平面区域的边界线和空间轨迹线均采用 NURBS 曲线表示, 扫成曲面则采用 NURBS 曲面逼近表示。

- (1) 确定步长, 确定关键点列;
- (2) 确定关键点处的轮廓线;
- (3) 以所有轮廓线的控制顶点构造 NURBS 曲面

(c) 平面区域变化规则地表示

- 显式表示 $C'(u, s) = f(s)c(u)$ (s : 弧长参数)
- 约束表示: 使二维轮廓线上的关键点满足一定的约束。例如, 在输入几条用于控制二维轮廓的形状。如右图。

3. 蒙皮操作 (skinning)

通过构造一系列封闭的二维轮廓线并在它们上进行蒙皮, 构造三维物体。

实现:

多边形轮廓: 与平移 sweeping 的拓扑构造方法相似。几何已基本给定, 要求截面平行。

带有曲线的轮廓: 采用 NURBS 曲面插值技术构造蒙皮面, 再加前后两底面。

蒙皮操作与广义 sweeping 操作的区别:

- (1) 蒙皮操作精确插值所有关键轮廓, 广义扫成是近似形状。前者适合于反求工程、仿制, 后者适合于创新设计。
- (2) 蒙皮操作的输入条件比广义扫成复杂。

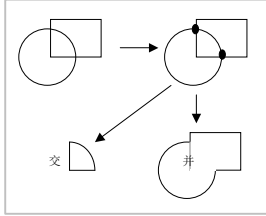
第五节 基于边界表示的集合运算

1. 算法分析

问题:

输入: 给定两物体的 B-reps, bA , bB

求: 结果 (拼合) 物体的边界表示 B-reps, $b(A \text{ <op> } B)$



可见,

- (1) 结果物体的边界是 A, B 两物体的边界的子集。
- (2) 具体的组成与具体的运算有关。

又如右图, 有二个二维的形体 A, B, 它们都是封闭的且维数一致, 则 A, B 可表示为 $A=(bA, iA)$, $B=(bB, iB)$ 。

我们先做 $A \cap B$ 。讨论一般集合运算的情况:

$$Q=A \cap B=(bA \cup iA) \cap (bB \cup iB)$$

$$=(bA \cap bB) \cup (iA \cap bB) \cup (bA \cap iB) \cup (iA \cap iB) \quad (1)$$

可见, 此运算产生了悬边, 显然是维数不一致。

因已知 $Q=bQ \cup iQ$, 我们一定能找到 bQ 和 iQ 的子集, 构成封闭且维数一致的形体 Q^* , 即 Q^* 一定能从 (1) 式导出。

由图(b)可知, $iA \cap iB = iQ = iQ^*$, 因此需要确定 bQ^* , 此处 bQ^* = 有效的 $(bA \cup bB)$ 。

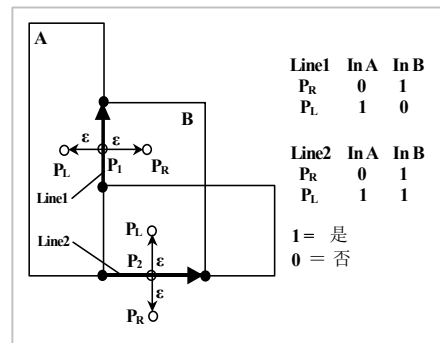
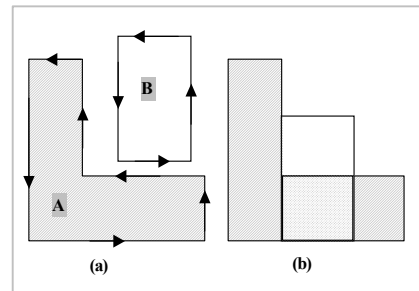
我们知道, 结果物体的边界是 A, B 两物体的边界的子集, 亦即任何新物体的边界是由组合单元的边界线段组成的; 边界点会变成内部点, 而内部点自身不会变成边界点。

对正则集的求交运算我们还会发现: $iA \cap bB \subset bQ^*$; $bA \cap iB \subset bQ^*$ 。

因此我们只需进一步分析 $bA \cap bB$ 。

正则集合运算下的边界测试:

方法 1: 邻域测试法。



方法 2: 边界切矢测试法。利用被测试边界的切矢。在上述 1, 2 两条边界中的 p_1, p_2 点处, 可以分别得到它们的切矢。在同一点处, 若两条边界线段的切矢同向, 则相应的边界为有效边界; 反之, 若两条边界线段的切矢异向, 则相应边界为无效边界。

因此对于求交运算的结果可表为:

$$Q^*=A \cap^* B=bQ^* \cup iQ^*$$

$$Q^*= \text{有效的} (bA \cap bB) \cup (iA \cap bB) \cup (bA \cap iB) \cup (iA \cap iB) \quad (2)$$

结论:

$$b(A \cap^* B) = b(A) \text{ in } B + b(B) \text{ in } A + bA \cap bB \text{ 同向部分} \quad (1)$$

由上述分析可知, 拼合操作的关键在于依据具体的运算确定相关的三部分原物体边界的子集, 而确定边界子集的关键又在于求边界面之间的交线, 因此算法也正是从求交线开始。

2. 算法步骤

算法主要由三部分组成：

- 求两物体各表面之间的交线段；
- 将所有有交表面沿交线剖开，并对每一部分进行面体分类；
- 建立新物体的数据结构。

1) 求物体各表面之间的交线段。

a) 包围盒初判，剔除无交面对 (enclosing box) (也曾有人使用过包围球)

- 需要建立体、面的包围盒 (与坐标系相关)
- 判别体体、面体、面面包围盒之间是否相交，若包围盒间无交，则物体间亦无交。

以 AABB 为例。

A: $X_{Amax}, X_{Amin}, Y_{Amax}, Y_{Amin}, Z_{Amax}, Z_{Amin}$ 为 A 的 box 表示

B: $X_{Bmax}, X_{Bmin}, Y_{Bmax}, Y_{Bmin}, Z_{Bmax}, Z_{Bmin}$ 为 B 的 box 表示

如果 $X_{Amax} < X_{Bmin}, Y_{Amax} < Y_{Bmin}, Z_{Amax} < Z_{Bmin}, X_{Bmax} < X_{Amin}, Y_{Bmax} < Y_{Amin}, Z_{Bmax} < Z_{Amin}$ 之一成立，则包围盒无交。

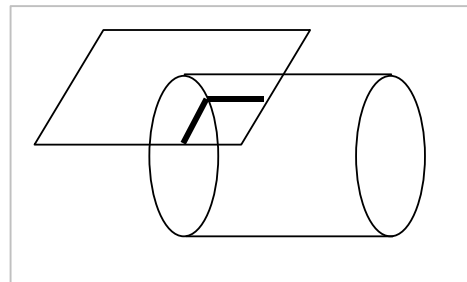
b) 求包围盒相交的面对之间的交线段

- 两曲面方程求交，求出交线方程。(对重合面需作特别处理，求出重合区域)
- 交线与每一面的边界求交，求出位于该曲面内部的部分。
- 求两曲面内部交线段的公共部分，如存在即为所求交线段 (参数法排序)。

2) 将所有有交表面沿交线剖开，并对每一部分进行面体分类

a) 对每一有交表面，搜索出其所有的交线段，连成交线。

- 所谓交线，即表面上一列首尾相连的交线段，或封闭，或从一边界线到另一边界线。
- 交线可有多条。
- 有交面一定为其与体的交线剖分成两部分。
- 面面之间为交线段，面体之间为交线。



b) 对剖分出的面进行面体分类，确定其是否属于结果物体 (与具体的布尔运算符有关)。

面体分类算法

3) 建立新物体的数据结构。

出发点：应尽可能利用已有数据结构，是对原数据结构的改造，而不是完全重建。

- 建立交线的数据结构，修改交面的数据结构。
- 重新建立结果物体的面表：交面中属于结果物体的新面 + 非交面中属于结果物体的面。
- 对于 $A - B$ 操作，还需对拼合体中属于原 B 物体的面作反向操作。

基于点集的关系分类

(1) 给定一点 P 及任意多边形 Q，求 P、Q 的分类。

- 射线法
- 弧长法 (角度法)

(c) 矢量法（针对凸多边形）

(2) 给定一点P及任意多面体Q，求P、Q的分类。

(a) 平面法

(b) 面积法

3. 一种多面体拼合运算算法

多面体拼合运算是拼合运算中最简单、最成熟的。这是因为多面体只包含一种曲面类型：即平面多边形，并且平面多边形之间的求交最为简单，平面多边形的交线分割后仍为平面多边形，表示十分统一。

连续求交法：

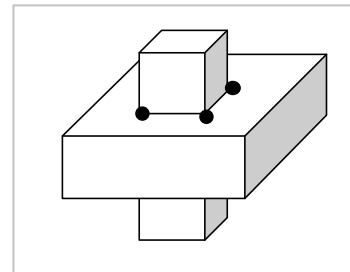
首先求出一个初始交点，然后从初始交点开始，采用当前交点的两特定相交面进行求交，求出交线段及下一交点，如此循环直至回到初始交点，形成一个封闭的交线环结束。

优点：

- 交线自然形成，避免了交线段重组。
- 易于在求交的同时建立起交线的数据结构。
- 在初始交点确定后，求交限制在相交面上。

问题：

- 首点的确定及多交线环的处理计算量很大。
- 对特殊情况处理困难。



多面体拼合运算算法：

2) 边面求交，建立交点表。

a) 边、多边形求交：设边为E，多边形为PolygonA

- 首先用E的方程与PolygonA所在的平面方程求交
- 如有交点，则再将交点关于E和PolygonA分别进行分类。如果交点同时位于E和PolygonA的内部和边界上，则交点为真正交点。

b) 建立交点表

由于多边形之间的交线段都是直线段，完全由两交点决定，即从交点可以找到相关的交线段，并且可以一次性生成交线。当然，要能够在全部的边、多边形求交结束后，基于交点搜索出交线，则必须建立交点结构，存放一些后面所需的信息。

对求出的交点，建立交点结构，放入一张表中。交点结构中包含下列信息：

(1) 交点本身；(2) 交边指针；(3) 交面指针

3) 搜索交点表，跟踪交线（对每一面分别进行。进行首尾相接的判别比较…）

a) 正常交点与交边、交面之间的关系

b) 跟踪交线算法

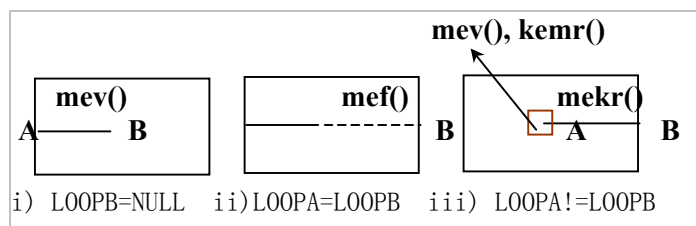
4) 建立交线的数据结构

从交线跟踪过程不难看出，每跟踪出一条交线段，便已经知道了该线段的以下信息：交线段的端点、形成交线段的两个面、交线段的前一线段、交点所在的边界。这些信息使我们能够在跟踪出交线的同时就建立起交线的数据结构，从而可以提高算法的效率。

交线数据结构的生成仍可采用Euler操作（可简介Mantyla的方法）。

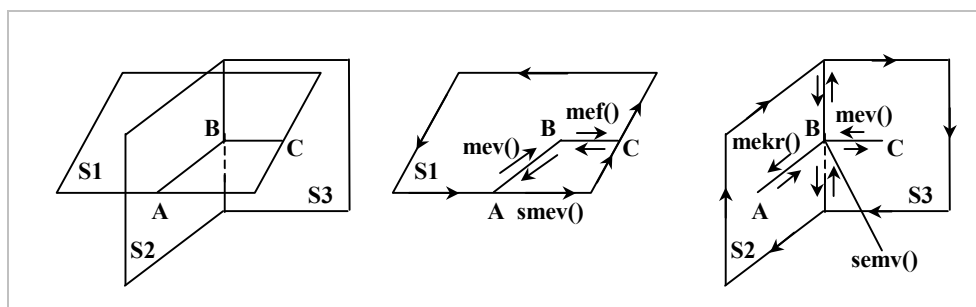
具体算法：

- a) 对交线段进行分类。每搜索出一段交线，首先根据交线两端点在每张面中的几何、拓扑性质，将其分为三类：



- b) 根据分类，采用相应的欧拉算子建立交线段的数据结构
 i) mev(); ii) semv(), mef(); iii) semv(), mekr();
- c) 对在两张面中新生成的 4 条半边，根据给定的布尔运算算子建立起它们之间新的邻边关系：
- 对于 U^* , $\cap^*$ ，反向边互为邻边
 - 对于 $-^*$ ，同向边互为邻边

Note: 交线段数据结构生成在两相交面中同时进行。



- 5) 面体分类：跟踪完一条封闭交线环进行一次面体分类。
- 6) 根据面体分类确定的一个面，搜索出结果物体的面表，并对 $A -^* B$ 中属于 B 中的面进行反向操作。

基于边界的拼合运算是十分复杂的算法，内容十分丰富，也是最耗时，最不可靠的操作。前面讲的多面体拼合运算算法，即使这个算法仍无法完全可靠，当加入曲面以后，问题还更多，主要是曲面求交和裁减曲面的问题，这些问题我们后面还会专门将。

第六节 局部操作 (Local Operation)

1. 概述

局部操作：指仅涉及对三维实体的几何信息或拓扑结构作局部变动的操作。

2. 局部操作

2.1 负操作 (Negating)

功能：将实体所有边界面的法向改为相反方向。

算法实现：改变实体中所有半边的走向。

2.2 扭变操作 (Tweaking)

[前提条件: 拓扑结构不变]

功能: 通过对一个物体的一部分表面实施几何变换或者改变某一表面的类型 (如由平面改为曲面) 实现对物体某一部分形状的改变。

算法实现:

- a) 对扭变表面作变换
- b) 对扭变表面的边界进行分类, 再依类进行处理:
 - 对内部点的坐标, 内部边、公共边的方程直接实施变换。
 - 边界边的方程置为与其相连的扭变面与非扭变面的交线方程。
 - 公共点的坐标由公共边与公共点的非扭变邻面求交得出。
 - 边界点用邻边与相关的扭变表面求交, 重新定位。

2.3 对称映射 (Reflecting)

功能: 通过对给定实体关于一个平面作反射变换得到另一个物体或改变给定实体的形状。

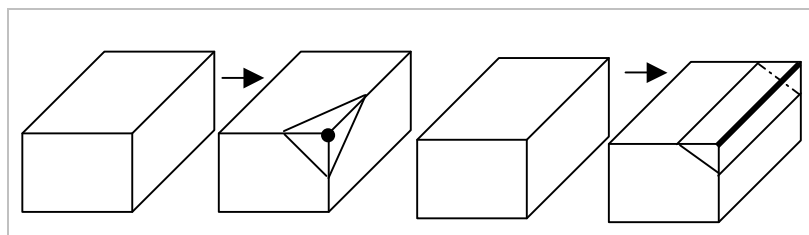
用途: 简化对称物体的构造。

算法实现:

- a) 在拓扑上, 复制原实体并对其作负操作。
- b) 在几何上, 使复制体中的点、边、面为原实体中点、边、面的对称元素。
- c) 如果给定平面为给定物体上的一个面, 则应取消不必要的点、线、面。

2.4 倒角 (Chamfering)

功能: 将物体的一个点 (或一条边) 用一小平面代替, 削去一尖角 (或一条棱边)。



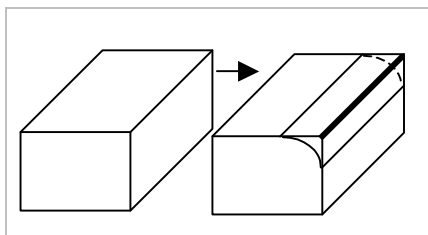
算法实现:

拓扑上: 用 `semv()`, `mef()` 生成新的点、新的边、新的面; 再用 `kef()`, `kev()` 删除不需要的点边、面。

几何上: 通过给定的参数计算出新的顶点坐标, 再基于新点导出新面的方程。

2.5 圆角 (Rounding) [磨光]

功能: 将物体上的一个点或一条边用曲面代替, 且保证新曲面与原点、边的相邻曲面之间光滑 (G^1) 拼接。



算法实现：

拓扑上：与倒角相似，用欧拉操作构造。

几何上：十分复杂，有多种方法：

- 割角法：用大量平面片逼近表示圆角曲面，实现方法是反复倒角。
- 二次曲面、双三次 Bezier 曲面、Gregory 曲面片表示。

2.6 粘合操作 (Gluing)

功能：将两个只在表面相交的实体沿它们的重合表面“粘合”成一个新的实体。

用途：实现搭积木的功能。

算法实现：

1) 确定贴合面的表示

- 拓扑上： $(A-B)$ 与 $(B-A)$ 的边界分别作为两个面表示（重合边不再作为贴合面的边界）
- 几何上：贴合面的边界为 $(A-B) \cup (B-A)$ 的边界。

2) 建立新物体的面表

- 对于重合边相连的两侧面进行判别（若重合，则删除重合边，合并重合面；不重合，则删除重合边，置两个侧面为相邻面）
- 将所有面的体指针 (fsolid) 指向新体。

2.7 扫成操作

功能：基于实体上的一个二维区域进行扫描，以改变实体的形状。

3. 特点

局部操作的特点：

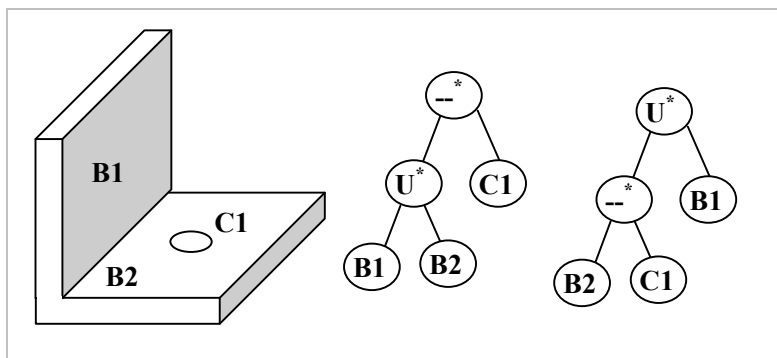
- 1) 效率高、使用方便，是有效的造型手段。
- 2) 不统一、不通用（从系统的角度看）。
- 3) 操作的输入和算法实现均依赖于实体的边界信息，因此需要 B-reps 的支持。
- 4) 有效性需用户保证，即用户需输入正确的结果。

第七节 CSG 表示法 (Constructive Solid Geometry)

CSG 树表示法是又一重要的实体表示法。首先由美国 Rochester 大学 A. A. G. Requicha 研究小组提出，其基础是实体的点集模型。

1. CSG 树的定义

CSG 树是用以表示三维实体的一棵二叉树，树的终端结点表示构造该实体的基本体素，中间结点表示正则集合运算。



2. 基本体素的种类及表示

2.1 基本体素的种类

设计多少种基本体素由系统的几何覆盖域，造型的方便性（希望尽可能多）、系统的复杂性（希望尽可能少）决定。

一般包含：长方体、圆柱体、圆锥体、球、正棱柱、正棱锥、环、楔形物、1/4 圆柱体等。

在纯 CSG 表示的造型系统中，平移 Sweeping，旋转 Sweeping 不可能成为基本体素。

2.2 基本体素的表示

在实体的 CSG 表示法中，基本体不采用边界表示，而是通过其边界的半空间进行表示。

半空间的概念：设面的方程为 $f(x, y, z) = 0$ ，则

$$H_-(f) = \{P(x, y, z) \in E^3 \mid f(P) \leq 0\}; \quad H_+(f) = \{P(x, y, z) \in E^3 \mid f(P) > 0\}$$

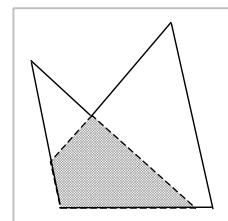
构成面的两个半空间。因为 $E^3 = H_-(f) \cup H_+(f)$ ，故称 H_- 、 H_+ 为半空间。

- 任一凸的三维实体可以表示为其边界面（为多项式曲面）半空间的交集

$$O = \bigcap_{i=1}^n H_-(f_i)$$

但对凹的实体，或包含自由曲面的物体，该结论不一定成立。

- 基本体素表示为其边界面的半空间的交集，因此所需要的是：面的方程。



2.3 CSG 树表示的数据结构

树结点的数据结构：

操作码	左子树	右子树	基本体素	坐标变换
-----	-----	-----	------	------

- 操作码：用于表示结点的性质，0 表示叶结点；非零表示中间结点。对于中间结点还通过值表示出具体的集合操作：1：并；2：交；3：差
- 左、右子树：是子结点指针，left-subtree, right-subtree，当结点的操作码非零时，左、右

- 子树非空，否则赋空指针。
- c) 基本体素：指向基本体素的指针。当操作码为零时非空；否则赋空指针。
 - d) 坐标变换：矩阵变换的指针，用于将中间物体变换到适当的位置。

2.4 CSG 树表示分析

- 记录实体的构造过程
- CSG 是一种隐式的描述
- 模型的构造简洁：用 CSG 树表示一个复杂形体非常简洁。它所产生的物体的有效性是由基本体素的有效性和集合运算的正规性自动保证的。
- 是完备的实体表示
- 问题：如何作局部操作？如何显示实体的边界？——>应用算法复杂

3. 分治算法 (Divide-and-Conquer)

3.1 分治思想

问题的求解（分解为）——>子问题的求解——>……>子问题解（合成）——>问题的解

CSG 树表示的分治算法的形式：

$F(\text{tree}, \text{data})$

$= \text{if}(\text{tree-is-a-leaf}) \text{ then } \text{Prim_F}(\text{tree}, \text{data})$

$\text{else } \text{combine}(F(\text{left-subtree}, \text{data}), F(\text{right-subtree}, \text{data}), \text{node}(\text{tree}))$

F 为计算某几何性质的函数， Prim_F 为计算基本体素几何性质的函数， combine 是对左右子树的结果进行合成的函数。

这里 Prim_F 易于设计，困难之处在于 $\text{combine}()$ 函数。

具体算法：

- 1) 边界生成
- 2) 图形显示
- 3) 物性计算（如体积）

上述算法的基础：集合成员分类算法 (Set Membership Classification)

3.2 点体分类算法(体为 CSG 树表示的物体)

(1) 点对基本体素的分类算法：半空间判别

(2) 分类结果的组合

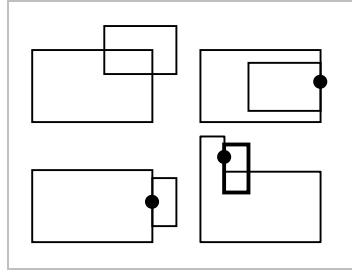
设点 P ，已知 $\text{PMC}(P, A)$ 和 $\text{PMC}(P, B)$ ， $S = A \langle \text{op} \rangle B$ ，

求 $\text{PMC}(P, S) = \text{combine}(\text{PMC}(P, A), \text{PMC}(P, B), \langle \text{op} \rangle)$

假定 $\langle \text{op} \rangle = \cap^*$ ，则 $S = A \cap^* B$ ，该算法可通过穷举法实现。

PMC(P, B)		In B	On B	Out B
PMC(P, A)	In A	In S	On S	Out S
	On A	On S	On S / Out S	Out S
	Out A	Out S	Out S	Out S

Out A	Out S	Out S	Out S
-------	-------	-------	-------



对 $PMC(P, A) = onA$, $PMC(P, B) = onB$ 的情况需作进一步的处理:

采用分治算法建立邻域函数, $NB(P, A \langle op \rangle B) = NB(P, A) \langle op \rangle NB(P, B)$

如果 $NB(P, S)$ 为空集, 则 P 为 S 的外点; 如果 $NB(P, S)$ 为满集, 则 P 为 S 的内点; 如果 $NB(P, S)$ 为非空非满集, 则 P 为 S 边界点。

3.3 边体分类算法

(1) 边对基本体素的分类算法

- 边与基本体素的所有面求交, 求出交点
- 判别交点是否位于边上、体上 (通过半空间), 确定出真正的交点
- 将边在真正交点处分段, 并判别出每一段的内外属性
- 最后返回分类结果.

$$EMC(X, A) = (X \text{ in } A, (X \text{ on } A, NB(X \text{ on } A, A)), X \text{ out } A)$$

(2) 分类组合:

a) 设 $\langle op \rangle = \cap^*$, $S = A \cap^* B$, X 边

已知:

$$EMC(X, A) = (X \text{ in } A, (X \text{ on } A, NB(X \text{ on } A, A)), X \text{ out } A)$$

$$EMC(X, B) = (X \text{ in } B, (X \text{ on } B, NB(X \text{ on } B, B)), X \text{ out } B)$$

$$\text{求: } EMC(X, S) = (X \text{ in } S, (X \text{ on } S, NB(X \text{ on } S, S)), X \text{ out } S)$$

$$X \text{ in } S = X \text{ in } A \cap^* X \text{ in } B$$

$$X \text{ on } S = t1 \cup t2 \cup t3, \text{ 这里,}$$

$$t1 = X \text{ on } A \cap^* X \text{ in } B$$

$$t2 = X \text{ on } B \cap^* X \text{ in } A$$

$$t3 = \{P \in X \text{ on } A \cap^* X \text{ on } B \mid NB(P, S) \text{ 既非空又非满}\}$$

$$X \text{ out } S = X \neg^* (X \text{ in } S \cup^* X \text{ on } S)$$

注意: $X \text{ out } S \neq X \text{ out } A \cup^* X \text{ out } B$

(b) 对于 $S = A \cup^* B$, $S = A \neg^* B$ 的情况

可转化为 $CA \cap^* CB$ 和 $A \cap^* CB$ 处理。

$$\text{即: } X \text{ in } (A \cup^* B) = X \text{ out } (CA \cap^* CB),$$

$$X \text{ out } (A \cup^* B) = X \text{ in } (CA \cap^* CB),$$

$$X \text{ on } (A \cup^* B) = X \text{ on } (CA \cap^* CB),$$

再利用， $X \text{ in } CA = X \text{ out } A$, $X \text{ out } CA = X \text{ in } A$, $X \text{ on } CA = X \text{ on } A$, 即可得出结果。

(3) 提高上述分类效率的手段

- 对于 A, B 均为凸体的情况，如果 $S = A \cap^* B \neq \phi$ (即不是仅相交在表面上)，则上述公式可以简化为：

$$X \text{ out } S = X \text{ out } A \cup^* X \text{ out } B$$

$$X \text{ on } S = D \text{ on } A \cup^* D \text{ on } B, \text{ 这里, } D = X -^* X \text{ out } S$$

$$X \text{ in } S = D - X \text{ on } S$$

提高效率的原因：省去了邻域函数的计算

应用范围：对基本体素分类结果进行初次组合时使用。

- 对 $X \text{ out } A = X$ 的情况 (即 X 完全位于 A 的外部)

S	EMC(X, S)
$A \cap^* B$	(ϕ, ϕ, X) (位于 S 的外部)
$A \cup^* B$	EMC(X, B)
$A -^* B$	(ϕ, ϕ, X) (位于 S 的外部)

- 利用包含关系

Relationship	EMC(X, S)
$S \subset R$, 且 $X \text{ in } R = \phi$	(ϕ, ϕ, X)
$R \subset S$, 且 $X \text{ out } R = \phi$	(X, ϕ, ϕ)

3.4 线框生成算法 (Wireframe Generation)

- (1) 生成所有可能的边界线 (所有基本体素的边界面之间进行求交)

将每一基本体素的所有边界面与其它基本体素的所有边界面进行求交，求出所有交线。

- (2) 确定真正位于体上的边界线。

将所有交线分别与 CSG 树表示的物体作边体分类，取其中位于体上的部分。

3.5 边界面生成算法 (Boundary Evaluation)

将每一基本体素的边界面 (无限面) 对 CSG 树表示的物体 S 进行面体分类，取 $f \text{ on } S$ 部分，即为物体的边界面。

3.6 光线投射算法 (Ray Casting)

用途：生成 CSG 表示物体的真实感图形。

做法：模拟照相的逆过程，从观察点 (视点) 出发向显示器屏幕的每一像素投射光线。经光线与场景中的物体求交得到光线与相距最近的可见表面的交点，并计算该交点处的表面法矢。最后根据光源的位置和景物的表面特性，利用光照明模型计算出像素的色彩和亮度，进而生成景物的真实感明暗图。

交点求法：

- 将射线与所有基本体素求交，求出位于其上的交点。

$$\text{光线方程: } X = X_0 + t * \text{Direction}$$

- 对所有交点进行点体分类，确定出位于体上的所有交点。

- 按参数 t 由小到大的顺序对交点的进行排序，确定出最近交

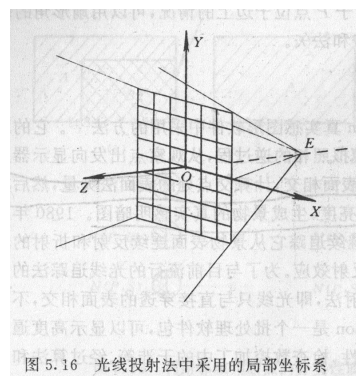


图 5.16 光线投射法中采用的局部坐标系

点。

可以得到下列两个数组：

$t(1), t(2), t(3), t(4), \dots, t(n-1), t(n)$ (参数)

$s(1), s(2), s(3), s(4), \dots, s(n-1), s(n)$ (表面指针)

显然,光线从 $t(1)$ 进入表面 $s(1)$, 从 $t(2)$ 退出表面 $s(2)$, 从 $t(3)$ 进入表面 $s(3)$, ..., 最后 从 $t(n)$ 退出表面 $s(n)$ 。 ($t(1), s(1)$) 离观察点最近。

$[0, t(1)], [t(2), t(3)], \dots$ 为体外线段

$[t(1), t(2)], [t(3), t(4)], \dots$ 为体内线段

光线投射算法不需要精确 CSG 问题的边界表示, 就可以直接生成物体的光栅图形, 为 CSG 表示法中的显示提供了有效的方法。

其它应用:

- (1) 计算体积
- (2) 生成消隐的线框图
- (3) CSG 树的集合运算过程

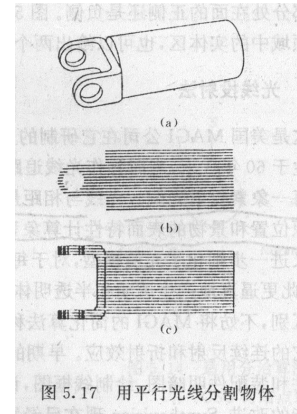


图 5.17 用平行光线分割物体

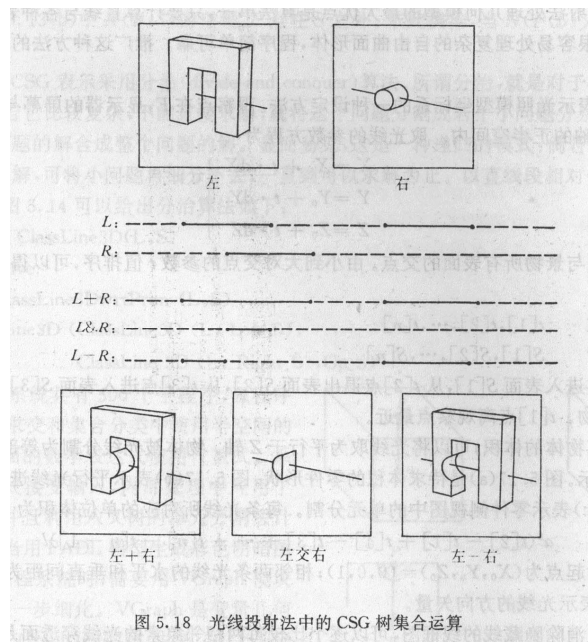
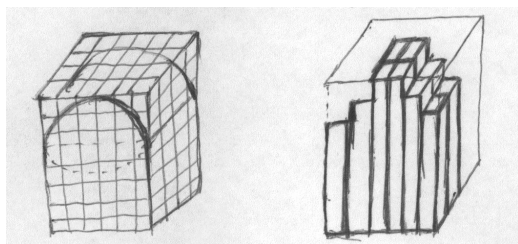


图 5.18 光线投射法中的 CSG 树集合运算

3.7 体积计算

通常采用以下的离散的方法:

- 1) 将 CSG 表示转化为空间分解表示



2) 计算单元体积的总和作为物体体积，是一种近似表示，近似计算。

第八节 分解模型 (Decomposition Model)

分解模型：将三维物体分解表示成相互粘合的单元体集合。

另一种含义：将三维空间分割成单元小空间的并集，则三维物体可以通过其占据的所有单元小空间近似表示。

分解模型的几何完备性取决于单元体的几何信息完备性。

2. 空间枚举 (Spatial Enumeration) :

单元体为具有同样大小与定向的立方体，立方体的面与世界坐标系的坐标面平行。

1.1 空间枚举表示的数据结构

一个三维整数数组 $C_{ijk}[][][]$ ，其中每一个元素代表一个单元立方体；元素在数组中的下标表示相应单元立方体左下角点的坐标；数组元素的值表示相应单元体与物体的关系：0——完全位于体外；1——完全位于体内；2——部分位于体内，或说包含物体的边界。

NOTE：需要时可增加其它属性：如材料

1.2 操作

● 构造：

方法之一：转换构造

由物体的 B-reps 或 CSG 表示转换成物体的空间枚举表示

方法之二：扫描输入

● 拼合操作：

d) 确定包含两物体的公共宇宙空间，及两物体在公共宇宙空间中的空间枚举表示，设为 C_{ijk} , C'_{ijk}

e) 通过比较 C_{ijk} 和 C'_{ijk} 中的相应值，并按下列规则合成出结果物体的表示 C''_{ijk} ：

并： $C''_{ijk} = 0$ ，如果 $C_{ijk} = 0$ 且 $C'_{ijk} = 0$ ；否则 $C''_{ijk} = 1$ ；

交： $C''_{ijk} = 1$ ，如果 $C_{ijk} = 1$ 且 $C'_{ijk} = 1$ ；否则 $C''_{ijk} = 0$ ；

差 (A-B)： $C''_{ijk} = 1$ ，如果 $C_{ijk} = 1$ 且 $C'_{ijk} = 0$ ；否则 $C''_{ijk} = 0$ ；

可见拼合操作十分简单、高效。

1.3 作用：

作为一种辅助模型，具有如下作用：

➤ 作为物体的网格剖分，支持有限元分析。

- 支持快速布尔运算，如干涉检查。
- 支持物性计算，如体积、重心等。
- 支持体绘制（Volume rendering）。

3. 空间分割（Cell Decomposition）

空间枚举法的问题：精度与存贮问题。

解决这个问题的方法是对宇宙空间采用自适应分割，而不是均匀分割。

三方向中分法：八叉树表示法

2.1 八叉树表示法

八叉树技术作为四叉树技术的推广，在计算机图形学中有着广泛的应用。空间八叉树算法是一个空间剖分算法，该算法将含有整个场景的空间立方体按三个方向中剖面分割成八个子立方体网格，组织成一棵八叉树。若某一子立方体网格中所含的景物面片数大于给定的阈值，则对该子立方体网格进行进一步剖分。上述剖分过程递归地进行直至八叉树中每一叶结点子空间所含景物面片数均小于给定阈值为止。

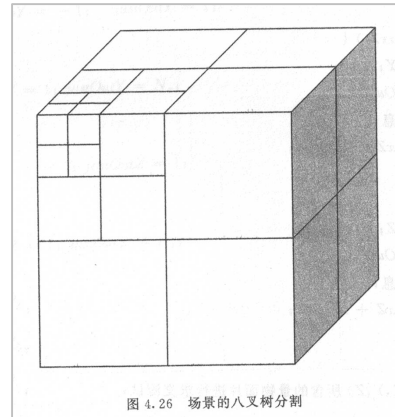


图 4.26 场景的八叉树分割

特点：单元体为同样定向，但可以不同大小的立方体，立方体的面与世界坐标的面平行。

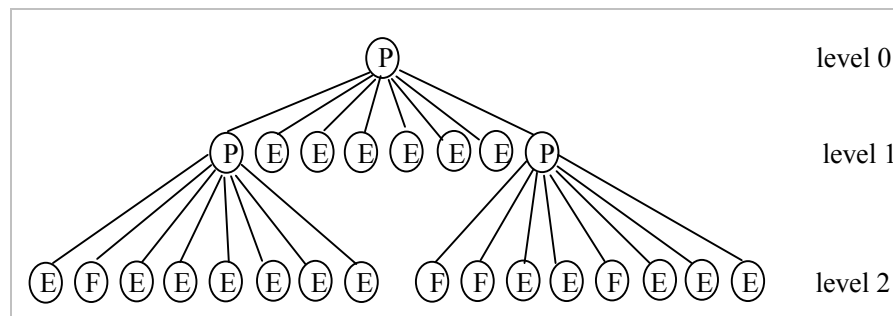
2.2 八叉树表示的数据结构

数据结构为一棵八叉树。树的根结点表示宇宙空间，其它结点表示分割后的一个单元体。

除根结点外，每个结点含八个子结点、一个父结点。每一结点具有一个属性值表示该结点所表示的单元体与物体之间的关系。即：

- 若物体完全占据这一子空间，则属性设为“Full”。
- 若物体部分占据这一子空间，则属性设为“Partial”。
- 若物体完全位于这一子空间外部，则属性设为“Empty”。

Example:



2.3 八叉树表示的构造

- 定义包含物体的宇宙空间，设为 $2^n \times 2^n \times 2^n$ ，生成树的根结点。

- b) 用宇宙空间的三个中分面将其分为八个子空间，每个子空间构成根结点的一个子结点，并依次建立每个子结点的属性：Full, Partial or Empty。
- c) 对于处于“Partial”状态的子空间，判别其是否满足精度要求，如不满足，继续对其施加 (b) 的过程，直到满足精度要求为止。

2.4 八叉树编码方式

单元体的几何信息通过树结点的编码提供。

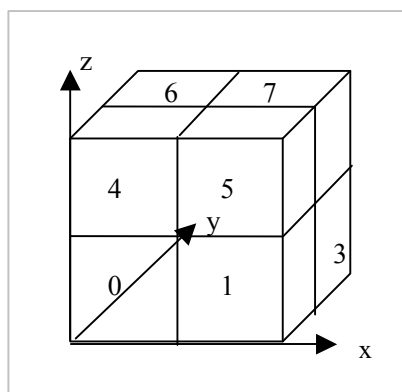
八叉树的编码方式如下：

1) 子结点的编号方式

- ✓ 含前左下角点的子结点编号为 0
- ✓ 沿 x 轴正向相邻兄弟结点编号增加 1
- ✓ 沿 y 轴正向相邻兄弟结点编号增加 2
- ✓ 沿 z 轴正向相邻兄弟结点编号增加 4

2) 任一结点的编码方式

为该结点在兄弟间的排行编号之前冠以父结点的编码，可表示为 $N_0N_1\cdots N_i\cdots N_m$, $0\leq N_i\leq 7$, m 为该结点在八叉树中的层次。



任一结点所表示单元体的几何信息：

设结点的编码为 $N_0N_1\cdots N_i\cdots N_m$, 则

- ✓ 边长 = 2^{n-m}
- ✓ 坐标原点 (x, y, z) (左下角点) 的计算方法如下：

$$\text{设 } N_i = e_i * 2^2 + d_i * 2^1 + c_i * 2^0, i = 0, 1, \cdots, m, \text{ 则}$$

$$x = c_0 * 2^{n-1} + c_1 * 2^{n-2} + \cdots + c_m * 2^{n-m+1}$$

$$y = d_0 * 2^{n-1} + d_1 * 2^{n-2} + \cdots + d_m * 2^{n-m+1}$$

$$z = e_0 * 2^{n-1} + e_1 * 2^{n-2} + \cdots + e_m * 2^{n-m+1}$$

2.5 八叉树表示的性质（分析）

- a) 八叉树表示是一种层次结构，每一层的全部非空结点给出了在某一精度下对物体的近似表示。
- b) 以简单形状的立方体表示非常复杂的物体，对这些物体的几何性质的计算及对它们的各种操作处理均可统一为对立方体的运算和处理，从而简单化。
- c) 由于八叉树表示给出了物体实际占据的空间，因此物体之间的和、差、交运算十分简便可靠，它们均为经典的树遍历算法。
- d) 八叉树算法便于并行处理。
- e) 八叉树表示可以表示物体内部的特性。
- f) 内存开销大，八叉树的结点数目同物体的表面积与最小立方体表面积之比成正比。
- g) 八叉树的树形和大小与所表示物体的位置有关。
- h) 八叉树表示丢失了物体的精确边界信息，例如曲面、和斜面的法向。
- i) 对其作变换十分复杂。

2.6 线性八叉树

定义：用八叉树终端结点编码的集合来表示一棵八叉树，为一可变长的一维数组。

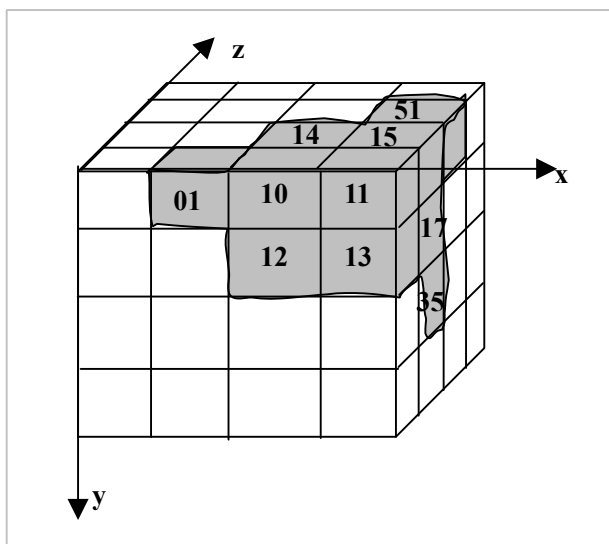
- ✓ 仅存贮八叉树的终端结点（仅显式表示最高精度的一层）
- ✓ 不包含空结点，也不标识 Partial 结点（作 Full 结点处理）→无法再提高精度

用途：克服八叉树存贮量大的问题。

举例：右图的线性八叉树编码表示为：

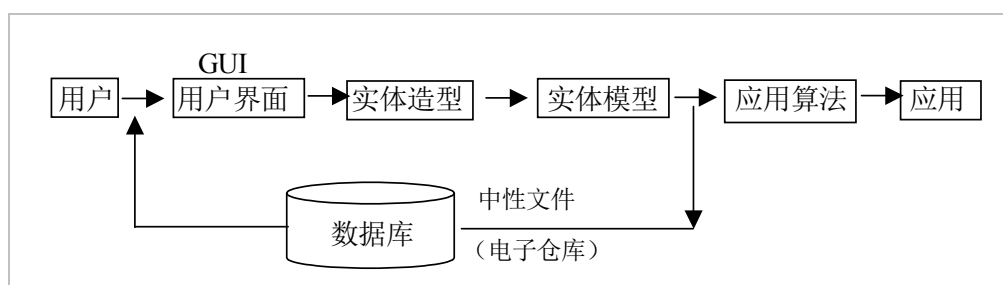
{01, 10, 11, 12, 13, 14, 15, 16, 17, 35, 51}

→{01, 1x, 35, 51}



第十节 实体造型系统

1. 系统构成（组成）



2. 系统设计

需要考虑的因素：好用、可靠、功能强、几何覆盖域宽、能有效支持各种应用

2.1 选择实体表示法

1) 选择的原则

原则之一：有效地支持建模

原则之二：有效地支持下游应用

2) 几种实体表示法的比较

B-rep: 造型（局部操作+sweeping 操作）、显示、基于约束的设计、CAM 刀具轨迹生成等

问题：如何支持修改，redo，undo 等功能；无体内信息。

CSG: 输入、修改+存贮

问题：隐式、难以直接许多下游应用、不唯一。

DM: 分析、计算、部分操作（如：相交检查）

问题：不精确。一般作为辅助模型。

3) 理想的方式

在造型系统中多种表示法并存，即混合（Hybrid）造型系统。

至少需解决两个问题：

- b) 不同表示之间如何转换（因为造型操作依赖于表示法）
- c) 不同表示之间的一致性如何保证（对某一模型作修改）

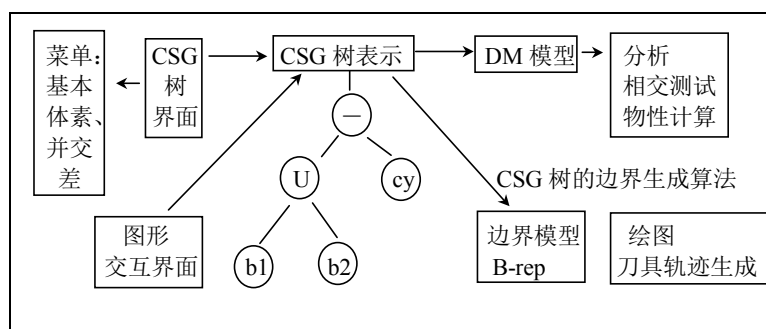
如：对 B-rep 的局部修改，如何一致性地传播到 CSG 和 DM 模型中

主要问题：系统规模将过于庞大

4) 实用的方式

以一种表示法为主，辅以其它表示

- a) 以 CSG 树表示为主，以 B-rep 表示为辅

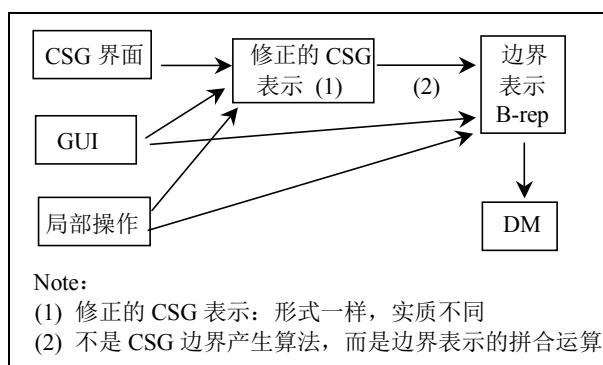


这类系统包括：PADL-I，PADL-II，GMSOLID

特点：

- 系统的造型操作是 CSG 表示的造型操作
- 系统的交互修改直接对 CSG 树进行
- 系统虽然包含边界表示，但该表示仅为内部表示，不与用户见面（用户无法选取）

- b) 以 B-reps 为主，以 CSG 表示为辅



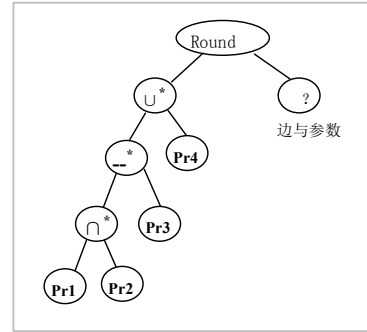
目前绝大多数的 CAD 系统的实体模型部分均采用如此的结构。

特点：

- 系统的造型操作是 B-rep 的造型操作
- CSG 树仅为形式上的 CSG 树表示，用以记录物体的构造过程和方便用户构造物体，修改

物体（如 undo），其与真正的 CSG 树表示有本质的区别：

- i) 基本体素的表示不同、种类不同：不是采用半空间表示，而是采用边界表示；种类丰富，含 sweeping 物体。（由于不需要半空间表示，并不增加系统太多的负担）
- ii) CSG 树的处理方法不同；
- iii) 中间结点的表示与种类亦不同。（可包含局部操作，如图）

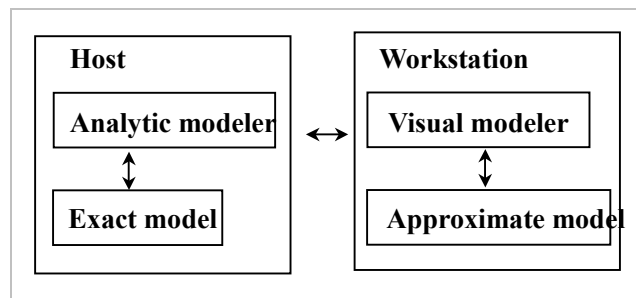


➤ 该混合表示通过对纯 B-rep 作简单修改（扩充）就可获得。

```
Struct Object {
    Object *father, *left, *right;
    Char name[MAX_NAME_LENGTH];
    INT ID;
    INT CODE; // operation
    FACE *f0; // first face in object
    TRANSFORM trans;
    PRIMITIVE *prim;
}
```

具体地，PRIMITIVE 包括 type, parameters, LCS, face 等信息。

c) 精确表示法与近似表示法的混合



2.2 确定系统的几何覆盖域和求交算法（实体造型系统的种类）

几何覆盖域：系统能够处理的几何形体的复杂程度

分类：a) 流行体 b) 非流行体

分类的关键：所能表示和处理的曲面的种类

2.2.1 流行体

1) 多面体系统

系统真正表示与处理的面为平面多边形，几何形体为多面体；界面中提供的曲面在系统中均被转化为多面体进行表示与处理。

优点：算法简单、可靠；

不足：精度问题、存贮问题。

2) 离散系统（facet system）

系统能表示平面与参数曲面，但核心操作（如拼合操作）仍为多面体算法。

离散系统与多面体系统的区别：

保存用户输入的曲面，并且多边形面有指向曲面的指针，用于记录离散小平面对应的原始曲面。多边形顶点包含其在原始曲面上的参数值。

优点：必要时可提高离散精度

问题：存贮更厉害、交线不光顺

3) 具有二次曲面功能的系统

系统能表示与处理平面和二次曲面，即传统机械工业零件。

复杂度体现在：

- a) 表示与处理（显示、求交等）二次曲线
- b) 表示与处理二次曲面（精确的二次曲面求交、Trimmed 二次曲面表示）

4) 具有雕塑曲面功能的系统

应用领域：飞机、汽车、船舶、模具工业

系统能表示与处理平面、二次曲面和自由曲面（NURBS），范围最广。

系统设计：

- a) 对平面多边形、二次曲面和自由曲面采用不同的方法，以各自的形式进行表示与处理。
- b) 对二次曲面和自由曲面统一表示和处理，对平面仍单独表示与处理。
- c) 将平面和二次曲面都统一成自由曲面进行表示和处理（SDRC、Master）。

上述三种设计各有优缺点，前者高效，但系统庞大；后者则相反。

复杂度体现在：

- a) 自由曲线的表示与处理（显示、求交）——>二维布尔运算复杂
- b) 自由曲面的表示与求交

NURBS 曲面：

$$S(u, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m w_{ij} P_{ij} B_{i,n}(u) B_{j,m}(v)}{\sum_{i=0}^n \sum_{j=0}^m w_{ij} B_{i,n}(u) B_{j,m}(v)}$$

2.2.2 非流形模型 (non-manifold model)

// 体的表面模型 本讲义的第二节中介绍

。

图 5.30 中的两个物体都不符合流形的定义。在实际处理中可以作出变通，将图 5.30(a) 看成是两个物体放在一起，而将图 5.30(b) 中的内孔和外周的共享边拆成两条边，面的尖角，另一条属于右面的尖角。

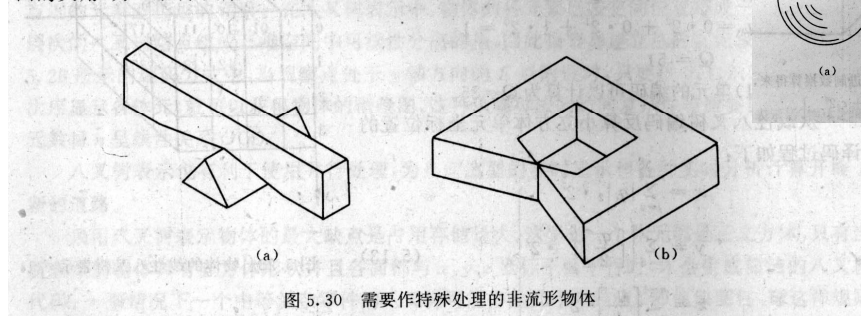


图 5.30 需要作特殊处理的非流形物体

非流形体引起集合运算算法上的困难，因此传统的实体造型系统都限定处理正常的情况，即要求形体是正规集，它的边界必须拓扑等价于流形。

实体造型技术从流形转向非流形，一方面有理论研究上的意义，但更重要的是出于工程应用上的实际需要。

CAD 杂志 1991 年第一期组织了“超越实体造型”的专刊，Requicha 等 1992 年发表实体造型技术综述，都是从机械行业的产品建模需要出发来讨论这一技术发展动向的。

1) 非流形体

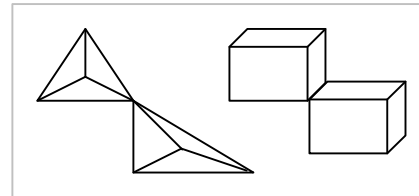
具有定向的曲面片族：

- a) 任一面均有确定取向，任两面规则相处
- b) 每一条边有偶数个面与其相关联，且以该有向边为顺向边的面数与以其为逆向边的面数相等

or:

指带有以下一种或几种几何元素的三维物体：

- a) 孤立点、孤立线、孤立面；
- b) 悬挂线、悬挂面；
- c) 迷向点（对应于多伞的顶点）；
- d) 迷向边（对应于多楔的边）。



2) 扩展的欧拉公式

$$V - E + F = 2(S - H) + R$$

$$(V - V_r) - (E - E_r) + (F - F_r) - I = 2(S - H)$$

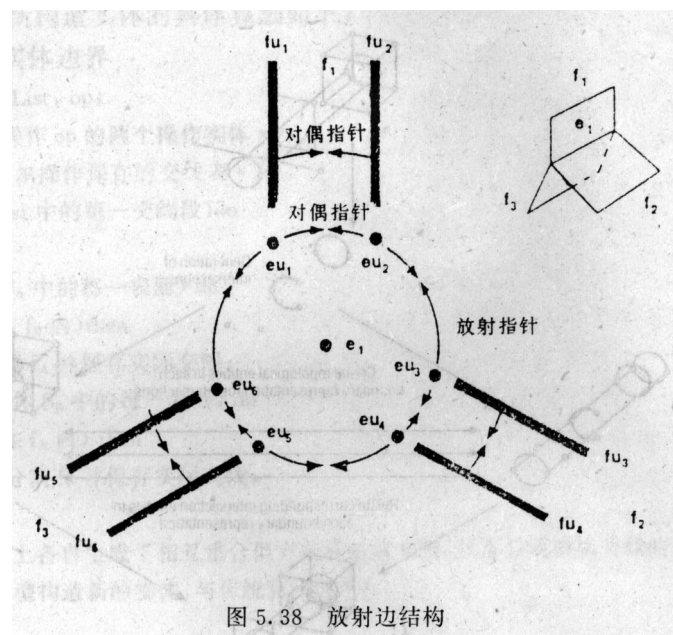
V_r : 与迷向点相连的伞数-1

F_r : 内环数

E_r : 与迷向边相连的楔数-1

I : 孤立的几何元素数目

3) 非流形体的边界表示数据结构



4) 多维非流形体的 SGC 模型 (Selective Geometric Complexes)

i) 定义

将非流形体表示为二维复形，具体由一系列的二维、一维、零维单形 (cell) 组成。

多维流形体：可以带悬挂线，悬挂面，孤立点、线、面的非流形体。

ii) 单形的数据结构

extent, 几何描述：方程、点的坐标

boundaries, 指低一维的边界元素或内部元素

star, 以该单形为边界的高一维的单形

active, 标识是否构成非流形体的边界

iii) 几个与复形相关的概念

两个复形相等：两个复形由相同的单形组成。

两个复形调和 (compatible)：设 A, B 为两复形，如果对每一个单形 $a \in A$, $b \in B$, 从 $a \cap b \neq \emptyset$ 可推出 $a=b$, 则称 A 与 B 调和。

复形细化：设 A, B 为两个复形，如果 A 的每一个单形都是 B 的某些单形的并，则 B 称为 A 的一个细化。

iv) SGC 的三个基本操作

➤ 离散：

输入：两个 SGC A 和 B

功能：通过离散 A 与 B 之间的重叠单形，对 A, B 进行细化，最后使 A, B 调和。

输出：两个调和的 SGC, A' , B' , 分别为 A, B 的细化。

插图

多维非流形体布尔运算举例。

➤ 选择

依据一定的准则，通过重新设置 active 标识符从一个或多个调和的 SGC 中选择出所需的单形。

➤ 化简

依据一定的规则，通过合并和删除单形，将一个给定的 SGC 转化成一个仍表示同样点集，但更为简洁的 SGC。

删除：删除 star 为空的 inactive 单形。

合并：合并具有公共边界的单形。

吸收：删除位于单形内部的低维单形。

插图

！有关多维非流形体的 SGC 模型将在后面补充。

2.2.3 非流形模型 (non-manifold model) // 补充版本

1) 非流形体

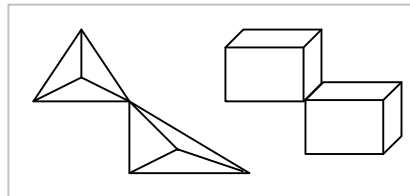
具有定向的曲面片族:

- c) 任一面均有确定取向，任两面规则相处
- d) 每一条边有偶数个面与其相关联，且以该有向边为顺向边的面数与以其为逆向边的面数相等

or:

指带有以下一种或几种几何元素的三维物体:

- a) 孤立点、孤立线、孤立面;
- b) 悬挂线、悬挂面;
- c) 迷向点 (对应于多伞的顶点);
- d) 迷向边 (对应于多楔的边)。



2) 扩展的欧拉公式

$$V - E + F = 2(S - H) + R$$

$$(V - V_r) - (E - E_r) + (F - F_r) - I = 2(S - H)$$

V_r : 与迷向点相连的伞数-1

F_r : 内环数

E_r : 与迷向边相连的楔数-1

I : 孤立的几何元素的数目

3) 非流形体的边界表示数据结构

Kevin Weiler 1986 年 8 月在 Rensselaer 工业学院发表的博士论文“几何造型的拓扑结构”对非流形实体的结构表示和遍历算法作出了开创性的研究成果。他设定的目标是用单一的通用格式来表示线框、曲面和实体三种建模手段，允许在产品模型中存储任何几何信息，如中心线、剖切平面等。Crocker 和 Masuda 等人都承认他们是在 Weiler 的技术路线的影响下进一步开展工作的结果。Weiler 针对非流形中一条边可以有多个邻面的情况，提出采用放射边的结构。他在零件模型中不直接表示面、环、边和顶点，而是表示它们的使用，如上图。每个拓扑元素所对应的几何表示是唯一的，但是在不同的遍历中可以表现不同的取向。例如一个面可以遍历两次，一次取逆时针边环，面的法矢量外指，另一次取顺时针边环，面的法矢量内指。每遍历一次记录一个 face-use 标号，所以一个面允许有两个 face-use 标号。一条边允许被多个面遍历，如右图三个面共享一条边，共有六个 edge-use

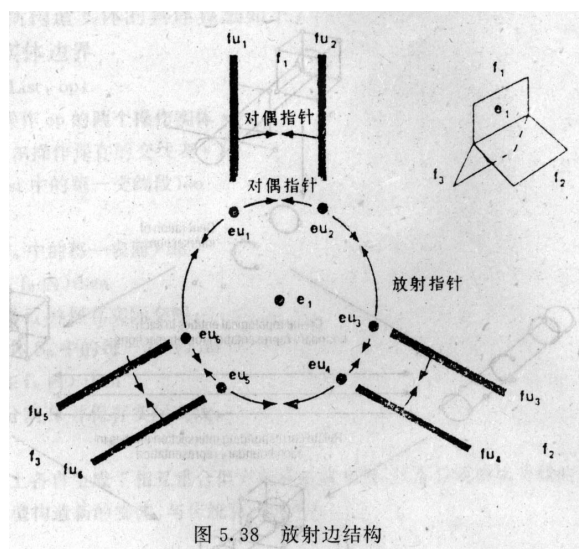
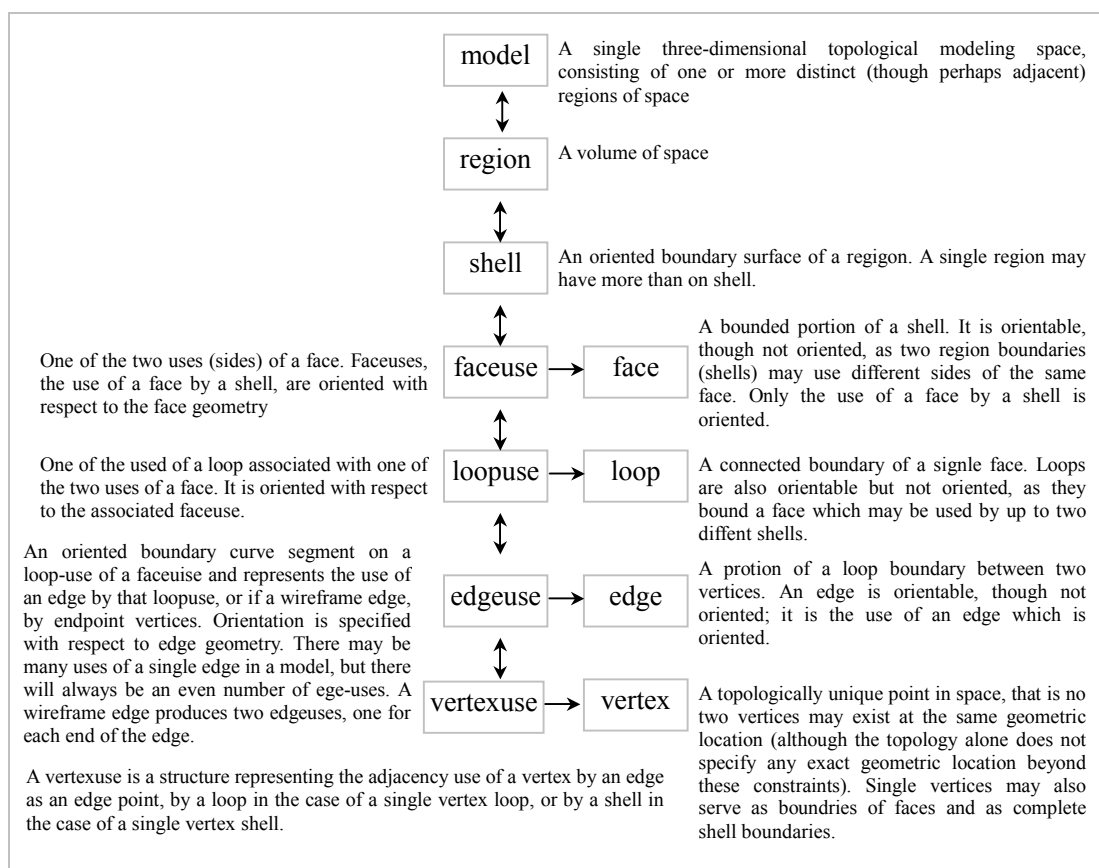


图 5.38 放射边结构

指针，相互间形成环形链。Weiler 还创建了更通用的扫成算法，点、线、面既可扫成高维的几何元素，即线、面和体，也可保持原来的维数，即仍是点、线和面，这样就可以用一次操作生成多样化的复合形体。



特点:

- i. 将 face, loop, edge, vertex 的几何定义与其使用 (use) 相分离, 使得模型中的不同拓扑元素可以共享相同的几何信息。从而保证非流形体数据的一致性。
- ii. 全面考虑了一般非流形体的情况, 实现了用统一的数据结构表示非流形体的目标, 提供了比正则形体表示更强的形体定义功能, 为实现更加简单而统一的算法奠定了基础。
- iii. 作为非流形体的边界表示, 其前提是所表示的形体具有边界, 它们可以处理几何元素不同维的情况, 但对开集或非封闭的点集不具有描述能力。
// 从表面拓扑的观点, 非流形形体是一个由 0、1、2、3 维几何元素组成的混合维形体, 它可以具有悬边、悬面和孤立点。以点集的观点, 非流形体可能具有内部边界或不封闭的边界, 也可以是开的点集。

3) SGC 模型(Selective Geometric Complexes)

为能表示任意维数的非封闭点集, J.R. Rossignac 和 M.A. O'Connor 提出了一种非流形体的分解表示方法——SGC (Selective Geometric Complexes, 选择几何复形) 模型。该方法能表示不同维数的物体和物体的内部结构。

基本思想: 将非流形体按维数进行分解, 同维间进行运算, 再将不同维数间的几何元素聚合, 具有分解方法的特点。

i) 定义

一个 SGC 模型由有限个互不相交的单形组成, 每个单形是连通的 n 维流形的开子集, 是点、边、面概念的扩展。亦即将非流形体表示为二维复形, 具体由一系列的二维、

一维、零维单形 (cell) 组成。

ii) 单形的数据结构

extent, 几何描述: 方程、点的坐标

boundaries, 指低一维的边界元素或内部元素

star, 以该单形为边界的高一维的单形

active, 标识是否构成非流形体的边界

- (1) 每个 SGC 单形都具有一个标识, 它规定了该单形是否处于“active”状态。同时 SGC 单形可以附加属性信息, 所有具有相同属性的“active”单形的并集就形成了一个区域(region)。SGC 将一个非流形体表示为所有“active”单形的并集。
- (2) 例如, 一个面(face)由一个开的曲面 M 和其边界的边和顶点的集合组成, 这些单形的并是闭的。每个单形的边界也是一组单形的集合, 所以几何复形表示的点集是闭的。
- (3) 可通过选择某些单形为“active”状态, 来表示各种点集。例如, 一个面(face)的内部被激活, 而其边界上的边和顶点不被激活, 表示的点集是面(face)的内部, 是开集。
- (4) 因此 SGC 可以表示非闭的点集, 这样的结果称为选择的几何复形。

iii) 几个与复形相关的概念

- ✓ 两个复形相等: 两个复形由相同的单形组成。
- ✓ 两个复形调和 (compatible): 设 A, B 为两复形, 如果对每一个单形 $a \in A, b \in B$, 从 $a \cap b \neq \emptyset$ 可推出 $a = b$, 则称 A 与 B 调和。
- ✓ 复形细化: 设 A, B 为两个复形, 如果 A 的每一个单形都是 B 的某些单形的并, 则 B 称为 A 的一个细化。

iv) SGC 的三个基本操作

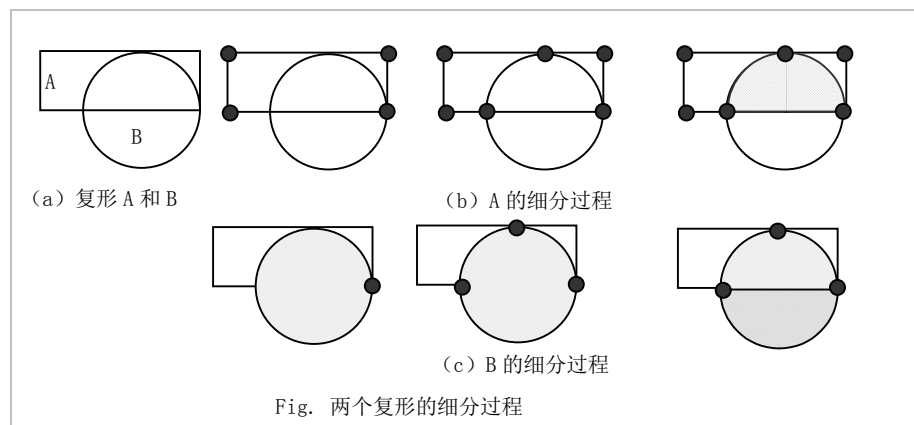
SGC 有三个基本算子: 细分 (subdivision)、选择 (selection) 和简化 (simplification), 作为基本操作, 其它布尔操作 (标准的集合运算) 和拓扑运算 (如求闭包、求内部、求边界等) 均建立在这三个基本算子的组合之上。SGC 方法可以表示任意的非流形体。SGC 通过 active 属性来表示物体的内部结构和开集。SGC 表示在标准的和正则的集合运算下封闭。

A) 细分 (subdivision): 将一个单形分解成几个单形, 使得物体与物体之间保持调和。

输入: 两个 SGC A 和 B

功能: 通过离散 A 与 B 之间的重叠单形, 对 A, B 进行细化, 最后使 A, B 调和。

输出: 两个调和的 SGC, A', B' , 分别为 A, B 的细化。

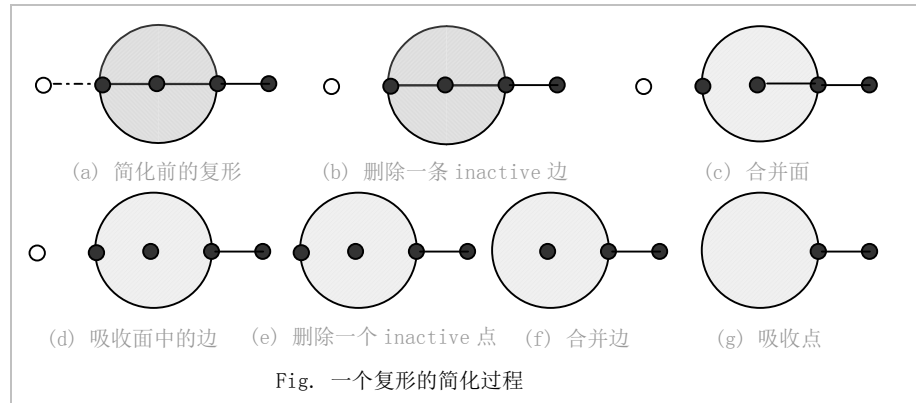


- B) 选择(selection): 定义某些单形为 active 状态。即依据一定的准则, 通过重新设置 active 标识符从一个或多个调和的 SGC 中选择出所需的单形。
- C) 简化(simplification): 依据一定的规则, 通过合并和删除单形, 将一个给定的 SGC 转化成一个仍表示同样点集, 但更为简洁的 SGC。

删除: 删除 star 为空的 inactive 单形。

吸收: 删除位于单形内部的低维单形。

合并: 合并具有公共边界的单形。



- D) 其它布尔操作 (标准的集合运算) 和拓扑运算 (如求闭包、求内部、求边界等) 均建立在这三个基本算子的组合之上。SGC 方法可以表示任意的非流形形体。SGC 通过 active 属性来表示物体的内部结构和开集。SGC 表示在标准的和正则的集合运算下封闭。

4) CNRG 表示法(Constructive Non-Regularized Geometry)

i) CNRG 对 CSG 的扩展

- (1) CNRG 使用标准 (非正则) 布尔操作和拓扑操作 (求边界、求闭包和求内部)。The regularized Boolean operations can be implemented in this scheme as three-operator sequences: standard, followed by interior and closure.
- (2) 允许 non-solid 物体作为基本体素, 如 point, curves, or surfaces, and higher-dimensional objects. (可能由多个 region 组成)
- (3) 引入了称为“聚合(aggregation)”的新的算子, 用于构造由多个区域(region)组成的结构物体(structured objects)。The aggregation operator is a formally-defined and more sophisticated version of the “assembly” operator provided by modelers such as PADL-2. Structured objects are not sets. They are collections of sets, much like the cell complexes of algebraic topology. Structured objects, also called CNRG objects, or simply objects, also have underlying sets, as cell complexes do. The underlying set of a CNRG object is the union of all the regions of the object, and therefore it is a set with no structure or “internal boundaries”.
- (4) CNRG 在 structured objects 上定义布尔和拓扑操作。

ii) CNRG 算子的语义 (Semantics of CNRG Operators)

A CNRG object is a set of pairwise disjoint possibly non-regular or disconnected sets, called **regions**.

A **region** is a possibly nonregular (semianalytic) subset of $\mathbb{R}^n$. Although a region of a CNRG object is a pointset and should be manipulated as a whole without further decomposition, for practical purposes, it may be internally represented in some modelers as the union of primitive geometric elements, such as vertices, edges, faces and other ‘cells’ as defined in “SGC: ... (Rossignac 89)”

The **pointset**, pA , of a CNRG object, A , is the union of the sets of its regions.

A CNRG tree is a rooted directed acyclic graph that represents a CNRG object. Leaves of the tree are CNRG primitives, which may be composed of more than one region. For simplicity, we assume that each primitive is given in its absolute position, although in practice, rigid body transformation nodes may be used. Internal nodes represent intermediate CNRG objects obtained by applying Boolean, topological, simplification, or filtering operators to the CNRG objects represented by their child-nodes.

To simplify the following discussion, we use “|” to denote a “gluing” or aggregation binary operator that takes two disjoint regions, aggregates of regions (all of which are pairwise disjoint), or disjoint CNRG objects and produces a CNRG object that aggregates all the regions. Note that expressions involving only aggregate operators are order-independent.

An uppercase letter denotes a CNRG object, and each of the object's regions is denoted by the same letter with a subscript. For example, A_1 and A_2 are two regions of the same CNRG object, A . If A is composed of only these two regions, we write, $A = \{A_1|A_2\}$. By definition, A_i and A_j are disjoint for $i \neq j$. A single-region object, A , is considered distinct from its region A_1 . We write $A = \{A_1\}$. Note that $A_1 = pA$ for single region objects.

In the following, we assume that S , A , B , C and D are CNRG objects. Furthermore, we often use C or S to denote the objects produced by applying to A (and to B) a Boolean or topological CNRG operator.

A and B are **disjoint**, if and only if the intersection of their sets, pA and pB , is empty.

Given two regions, A_i and B_j , $A_i \cap B_j$ denotes their intersection and $A_i \setminus B_j$ their difference in the standard set-theoretical sense. Note that $A_i \cap B_j$ and $A_i \setminus B_j$ are single regions that may be empty, disconnected, dimensionally inhomogeneous, and not closed.

A region is said to be **contained** in an object if it is contained in the set of the object. It need not correspond to the union of any subset of the object's regions.

We now define a set of new operators for CNRG objects. These operators are called: union, intersection, difference, complement, interior, closure, boundary and regularization. They are analogues to the standard pointset operators. We use a different notation to distinguish between the two set of operators. For example, we use ' $\cap$ ', ' $\cup$ ', and ' $\setminus$ ' for pointset intersection, union, and difference, and ' $*$ ', ' $+$ ' and ' $-$ ' for their CNRG counterparts. The topological complement of a pointset S is denoted $\bar{S}$. The corresponding CNRG operator denoted ' c '.

....

The aggregate operator ' $|$ ' and the pointset operator ' p ' were introduced in this paper to simplify the definitions. They need not be made available for use in CNRG trees. Instead, the union and simplification operators, precisely defined below, should be used. A filter operator that eliminates selected regions is introduced the next section. We define below the semantics for each of the CNRG operators.

Aggregation

Given n disjoint sets, A_i , the **aggregation** operation, denoted " $|$ ", creates the corresponding CNRG object: $A = \{A_1 | A_2 | \dots | A_n\}$.

Simplification

The **simplification**, sA , of A is a CNRG object with a single region: the set pA . Thus, $sA = \{pA\}$.

Complement

The **complement**, cA , of A is an object composed of a single region that is the set complement of pA . Hence, $cA = \{\overline{pA}\}$.

Union

The **union**, $A+B$, is an aggregate of regions of the following three types: $A_i \cap B_j$, $A_i \setminus pB$, and $B_j \setminus pA$, for all combinations of regions, A_i , of A and regions, B_j , of B . Potentially each region of A is split into two sets: the part in B and the part outside of B . The second set is a single region. The first set may be decomposed according to the decomposition of B into regions. Union produces a subdivision of $(pA) \cup (pB)$ that is compatible with the decomposition of A and B into regions. We call it "union" because the set $p(A+B)$ equals the set theoretic union, $pA \cup pB$.

Intersection

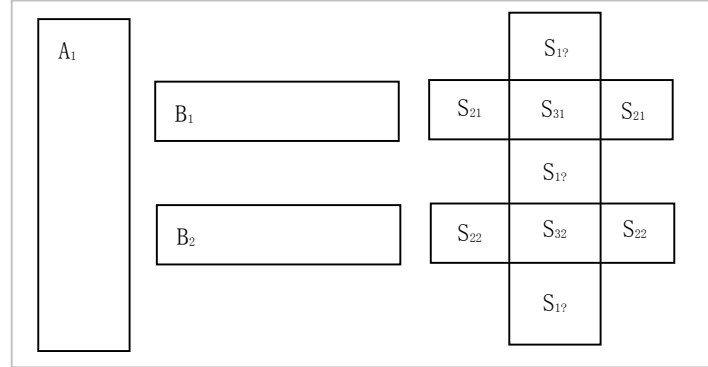
The **intersection**, $A * B$, of A and B is the aggregate of all regions of the type $A_i \cap B_j$. A region, A_i , of A is truncated to $A_i \cap pB$, and is subdivided according to the subdivision of B into regions. Consequently, $p(A * B) = (pA) \cap (pB)$, which justifies the name for this operator.

Difference

The **difference**, $A - B$, is the aggregate of all regions of the type $A_i \setminus pB$. Clearly, $p(A-B) = (pA) \setminus (pB)$.

It follows from the definition of union, intersection, and difference that $A + B = \{(A - B) \mid (A * B) \mid (B - A)\}$.

Union: a single-region object (left) A is unioned with an object (center) B that has two regions. In the resulting object (right), $S = A+B$, the unique region A_1 of A is split into



three regions: $S_{1?} = A_1 \setminus pB$, which is disconnected, $S_{31} = A_1 \cap B_1$, and $S_{32} = A_1 \cap B_2$. Each one of the two regions of B is split into two parts: their respective overlap with A (S_{31} and S_{32}) and the parts out of A ($S_{21} = B_1 \setminus pA$ and $S_{22} = B_2 \setminus pA$). The logic of the subscript notation is discussed below.)

Interior

The topological **interior**, iA , of A in $\mathbf{R}^n$ is the aggregate of regions, $A_i \cap \text{interior}(pA)$, that are the intersection of the regions of A with the topological interior of A in $\mathbf{R}^n$. Note that, although iA is always full-dimensional, regions of iA need not be full-dimensional. Thus the *interior* operator returns a subset of the original object and preserves its internal decomposition into regions. The set, piA , of the interior of A equals the topological interior of pA .

Closure

The **closure**, kA , of A , is the aggregate composed of all the regions of A plus a single new region defined as the difference between the topological closure of pA and pA itself. Thus, we can write: $k\{A_1, \dots, A_n\} = \{A_1, \dots, A_n\} \cup (\text{closure}(pA) - pA)$, and pkA equals the topological closure of pA .

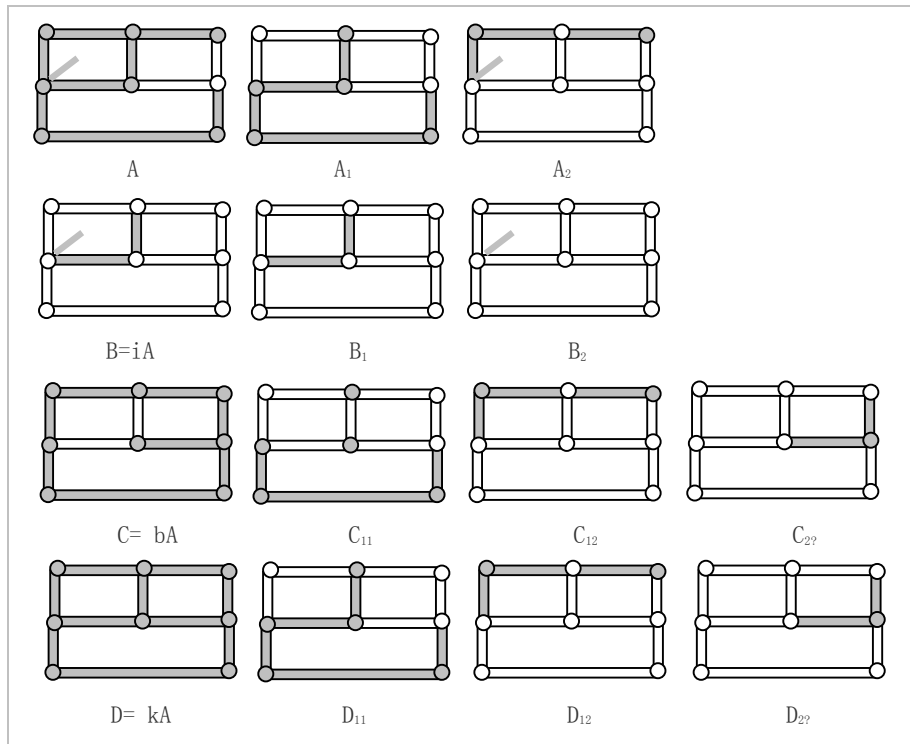
Boundary

The topological **boundary**, ∂A , of A is defined as: $\partial A = kA - iA$. The boundary operator does not necessarily return a single-region object. Note that $p(\partial A)$ equals the topological boundary of pA .

Regularization

The **regularized** version, rA , of A is defined as kiA . The set, prA , spanned by rA corresponds to a regularized solid as defined in [27] and equals rpA . Note that regularization does not imply simplification (i.e., a regularized object may be composed of many non-regularized regions). However, taking the boundary, C , of a regularized object, A , will produce a lower-dimensional object whose set, pC , is the topological boundary of the set of the regularized object, prA .

Operations: the top row shows a CNRG object A composed of two regions, A_1 and A_2 shown on the same line. The next row shows B , the interior of A , and its two constituent regions. The third row shows C the boundary of A (labeled 'bA') and its three constituent regions: a subset of A_1 , a subset of A_2 , and a new region, outside of A . The bottom row shows D , the closure of A , and its three constituent regions: A_1 , A_2 and the same new region as in C . Note that $p(kA) = p(iA) \cup p(\partial A)$. The subscript notation, called the region's signature, and our convention for constructing it, is introduced below.



Identifying and Selecting Regions

Assume that A and B are composed of m and n regions, respectively, and that C corresponds to a single-operator expression involving a left child, A (and possibly a right child B, for CNRG operators that take two arguments).

Union: The union $A + B$ has $(n+m+nm)$ regions of three different types. Each type corresponds to a different interpretation of '+', which we distinguish by associating the corresponding subscript number with the '+' sign:

- (1) Interpretation $A +_1 B$ yields the m subsets of the m regions of A that lie out of pB.
- (2) Interpretation $A +_2 B$ yields the n subsets of the n regions of B that lie out of pA.
- (3) Interpretation $A +_3 B$ yields the (nm) regions of type $A_i \cap B_j$, that lie in both A and B.

Intersection:

...

iii) Evaluation

- (1) Evaluation 的两种方法:

CNRG graphs may be evaluated directly or converted into an expanded SGC form, where SGC cells are grouped to form sets that represent CNRG regions.

- (2) 包含关系判别: (对类似于 CSG 树中的分治算法进行扩展)

Point inclusion test for a particular CNRG region may be carried out using an extension of the divide-and-conquer techniques popular with CSG trees.

- (3) 由 CNRG 表达式定义的非流形体的 SGC 表示的计算
简单地自下向上逐步执行 CNRG 操作即可。这些操作均可通过 SGC 的基本操作 (Subdivision, Selection, and Simplification) 的组合来构建。

CNRG models should be viewed as a primary model for user interaction because they support a high level vocabulary for expressing operations and regions, and because the CNRG trees are, similar to CSG trees, easy to edit and archive.

5) STC 表示法(Structured Topological Complexes)

J. Rossignac, Structured Topological Complexes: A feature-based API for Non-Manifold Topologies, Proc. of ACM Symposium on Solid Modeling and Applications, Atlanta, May 14-16, 1997.

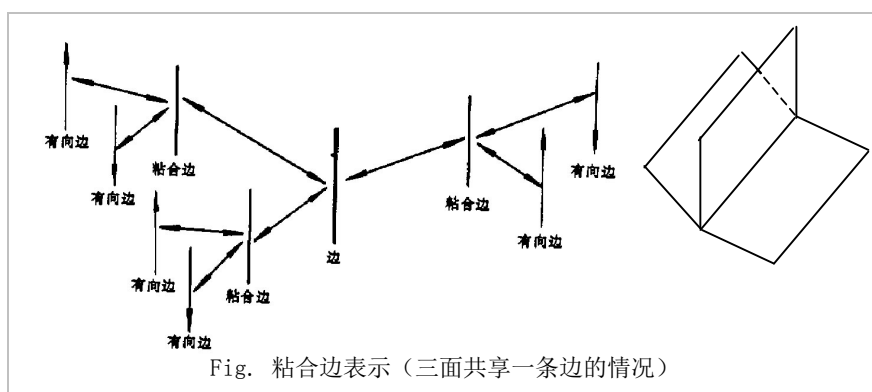
[Abstract] Much of recent research in representation schemes for solid modeling was focused on the extension of boundary representations to support non-manifold topologies. We introduce here a very simple, yet general formalism, which subsumes and simplifies many previous attempts at defining high-level operators for creating and interrogating such models. Our Structured Topological Complex (STC) extends our previous work on Selective Geometric Complexes (SGC) and on Constructive Non-Regularized Geometry (CNRG) and provides the foundations for a new generation of representations -- both constructive and evaluated -- and of APIs that are independent of the particular geometric domain and even of the particular approximation scheme for geometric primitives.

扩展了 SGC 和 CNRG，提供了新一代表示手段的基础。既是构造性的，又是显式的。

6) 国内的工作

中科院软件所武仲科、吴俊恒、戴国忠等

- (1) 将代数拓扑中复形的概念和方法引入到非流形表示中。他们给出了非流形的一个严格的数学定义，说明复形统一了三种模型，即零维骨架为全部顶点，一维骨架为棱集合加上悬点（线框模型），二维骨架为面集合、悬边和悬点（表面模型），三维骨架为三维实体（实体模型）。
- (2) 将 radial edge 数据结构加以扩展，提出了一种粘合边(adjunction edge)数据结构，其中每条边有多个粘合边，每条粘合边由两条有向边组成。



- (3) 操作分为基本操作和高级操作，基本操作包括生成单形、粘合及分离操作、扩展的欧拉操作等；高级操作包括扫描操作、驱动操作、分裂操作、布尔操作等。
- (4) 粘合边结构表示的拓扑元素之间的逻辑关系简洁、直观，并且可以表示物体的内部结构。
- (5) 粘合边结构是一种表示非流形体的方法，它建立在严格的数学理论之上，刻划的拓扑关系细致，建立的模型灵活、简洁、实现方便。

3. 系统的可靠性问题

到目前为止，所有的实体造型系统均存在不可靠的问题，只是程度不等。具体表现在：

- 1) 几何操作（造型操作）失败。
- 2) 产生错误结果。

而不可靠性以拼合运算为最。

3.1 实体造型系统不可靠的原因

1) 根本原因

计算机对实数采用的是浮点表示，而不是精确表示，从而具有计算误差。

从理论上说，判别两点是否重合的条件为 $\text{dist}(p_1, p_2) = 0$ ，但在算法处理上，却只能借助与误差 ϵ ，即 $\text{dist}(p_1, p_2) < \epsilon$ 。

ϵ 如何选择？若太大——>不重合点判为重合点；若太小——>重合点判为不重合。

经验作法是：把 ϵ 定义为机器精度的 100 倍。（Segal 提出）

2) 直接原因

首先，几何关系判别具有相对性。

(几何关系包括：平行、相交、重合、分离等)

在浮点数系下进行的几何关系判别，其结果是相对于所选择的容差而言的。

其次，几何关系具有相关性。

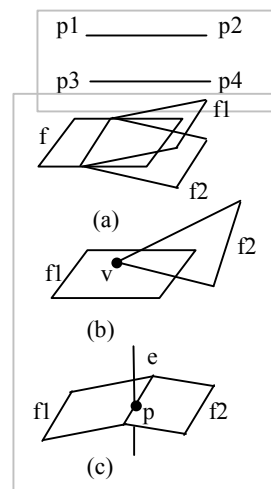
3) 结论

- i) 实数系能够保证相关几何关系的一致性。
- ii) 浮点数系下相关几何关系可能发生冲突。

例如：

- a) 面面重合关系之间的冲突
f 与 f1 重合，f 与 f2 重合，但 f1 与 f2 不重合。
- b) 点面重合关系与面面重合关系之间的冲突
f1, f2 共面，但 V 不在 f1 上。
- c) 点、多边形分类关系之间的冲突
P 既在 f1 外，又在 f2 外，导致交点丢失。

NOTE：如果 P 仅位于 f1 或 f2，问题同样存在，交线跟踪失败。



3.2 一种提高多面体拼合运算可靠性的方法（策略）

李新友，提高实体布尔运算可靠性的研究与实现，清华大学博士论文，1989 年 10 月。

1) 基本思想

- ✓ 对不能保证正确结果的情况能够判别出来，予以返回。
- ✓ 在进行布尔运算之前首先判别出所有的重合关系，并检查重合关系的一致性，如不一致，则进行调整，成功则继续进行物体之间的拼合运算，否则宣布不能做。

2) 算法步骤

i) 初步确定两物体之间的重合关系

- 重合关系的种类：点面重合 (PF)、点边重合 (PE)、点点重合 (PP)、边面重合 (EF)、边边重合 (EE)、面面重合 (FF)
- 系统给定一个容差 ϵ ，用以判别点面重合关系：

设点为 P，多边形面 F 为 {V1, V2, ..., Vn}, Vn+1 = V1, Vi(xi, yi, zi), F 的方程为 $ax+by+cz = d$ 。

$$a = \sum_{i=1}^n (y_i - y_{i+1})(z_i + z_{i+1}) \quad b = \sum_{i=1}^n (z_i - z_{i+1})(x_i + x_{i+1})$$

$$c = \sum_{i=1}^n (x_i - x_{i+1})(y_i + y_{i+1})$$

$$d_i = -(ax_i + by_i + cz_i - d)$$

取

$$tp = 2 * \max_{i=1}^n \{d_i\}$$

则如果 $|\text{dist}(P1, F1)| \leq tp + \epsilon$ ，则 P 与 F 有 PF 关系。

- 通过 PF 关系确定其它几种重合关系

1) FF 关系判别

如果 F1 的所有顶点与 F2 有 PF 关系，且 F2 的所有顶点与 F1 有 PF 关系，则 F1 与 F2 具有 FF 关系。

2) PE 关系判别

点 P 落在边 e 上的条件时 P 与 e 的所有邻面有 PF 关系。如图(a)。

3) PP 关系判别

两个点 P1 与 P2 有 PP 关系的前提是 P1 与 P2 的所有邻边有 PE 关系，同时 P2 与 P1 的所有邻边也有 PE 关系。如图(b)。

4) EF 关系判别

EF 关系是指物体 A 上有一条边 e 的两个端点 P1 和 P2 与物体 B 的平面 F 都有 PF 关系。如图(c)。

5) EE 关系判别

EE 关系是物体 A 上的一条边 e1 的两个端点 P1、P2 与物体 B 的一条边 e2 有 PE 关系，反之亦然。如图(d)。

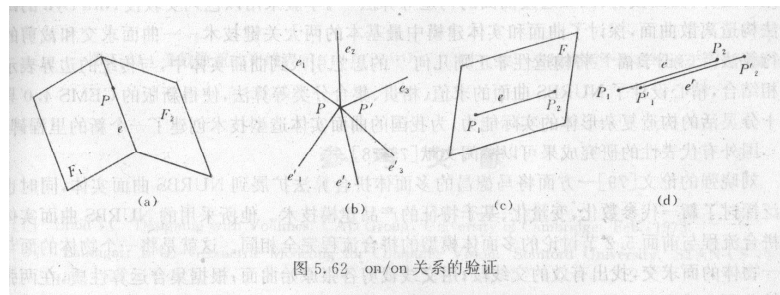


图 5.62 on/on 关系的验证

ii) 调整不一致的重合关系

Note: 基于 PF 关系推导其它五种重合关系时，能够发现重合关系的不一致性（这是如此设计的一个重要作用）。例如，F1 与 F2 的顶点某些具有 PF 关系，某些没有，因此无法判别是相交还是重合，F1 与 F2 的 FF 关系是含糊的。

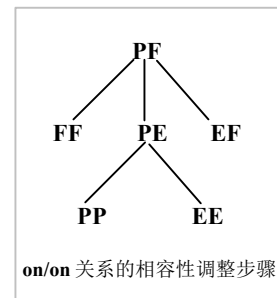
对上述情况，采用调整 ϵ 的方法，使 F1 与 F2 之间的 FF 关系变成明确的关系。

具体调整哪一种关系，通过计算两平面法向的点积进行判定：

$$\text{dot} = a_1a_2 + b_1b_2 + c_1c_2, \quad ||\text{dot}|-1| < \epsilon_{\text{dot}},$$

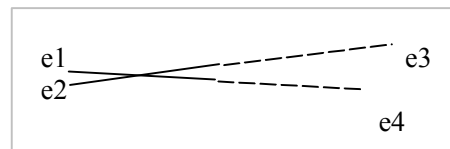
如此又依赖于一套新的容差 ϵ_{dot} 等。（是否可以通过以下原则确定调整为哪一类：如果重合的 PF 多，则调整为重合；否则调整为不重合？）

问题：缺乏有效的调整方法（因为又涉及到多种容差，并且如何设置调整步长？）、无法保证收敛、效率问题。



3.3 基于几何元素容差表示提高几何近似可靠性的方法

传统的方法：基于一个固定的容差进行几何关系判别，应该说不合理的。由于采用固定容差，因此判别出 e1 与 e2 重合，并且 e3 与 e4 重合。显然，判定 e3 与 e4 重合并不合适。造成这种情况的原因是现有方法允许误差积



累。

基本思想：a) 对于物体中的每一几何元素都采用容差表示，都具有自己特定的、合适的容差，并且 b) 基于几何元素的容差表示进行几何关系判别和不可靠性判别，c) 在判别出几何关系后动态调整几何元素的容差表示，以消除不可靠性。

i. 几何元素的容差表示

几何元素的容差表示由几何元素和它的三个容差域，即 ϵ 域、 δ 域和 Δ 域组成。

容差域的初始定义：

$$\epsilon = \tau - \gamma$$

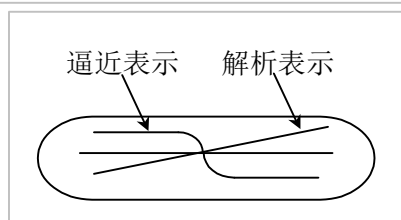
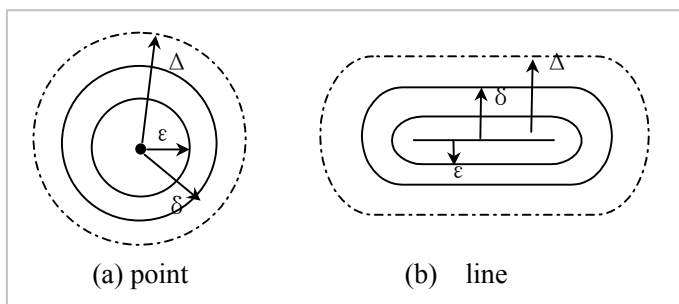
$$\delta = \tau + \gamma$$

$$\Delta = +\infty$$

这里 τ 为系统的总容差， $\gamma \ll \tau$ 为系统的二阶误差。

几何元素的解析表示（物体的解析表示）：位于一几何元素容差区域之内，并与该几何元素具有同样解析形式的几何元素成为该几何元素的一解析表示。

几何元素的逼近表示（物体的逼近表示）：位于一几何元素容差区域之内，满足该几何元素的所有约束，但解析形式不同的几何元素，称为其逼近表示。



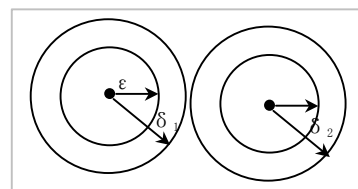
ii. 基于几何元素容差表示的几何关系判定与误差动态调整方法

几何关系种类：分离、重合、包含、相交、模糊 (ambiguity)

i) 分离关系

判别条件：若两几何元素 $O_1 (\epsilon_1, \delta_1, \Delta_1)$ 和 $O_2 (\epsilon_2, \delta_2, \Delta_2)$ 的 δ 域不交，则 O_1 与 O_2 分离。

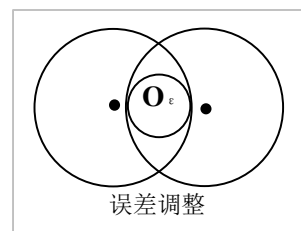
误差调整规则：设 O_1 与 O_2 间的距离为 d ，如果 $d < \Delta_1$ ，则 $\Delta_1 = d/2$ ($i = 1, 2$)。



ii) 重合关系（对于同维几何元素）

判别条件：若两几何元素 $O_1 (\epsilon_1, \delta_1, \Delta_1)$ 和 $O_2 (\epsilon_2, \delta_2, \Delta_2)$ 在它们的 ϵ 域的交集内存在一个它们共同的逼近表示，则 O_1 与 O_2 重合，即相同。

误差调整规则：将两个几何元素 O_1 与 O_2 的容差表示融合成一个几何元素 O 的容差表示。这里几何元素 O 为 O_1 与 O_2 的 ϵ 域交集中的一个与 O_1 、 O_2 同类型的几何元素。 ϵ 域和 Δ 域为 O_1 、 O_2 的 ϵ 域和 Δ 域交集的最大可能域； δ 域为 O_1 、 O_2 的 δ 域并集中的最大可能区域。



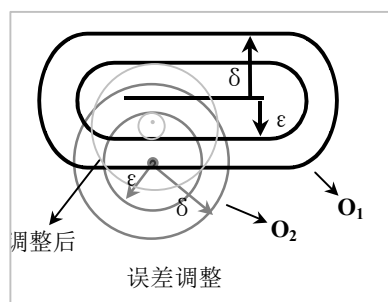
显然，这样可以解决误差积累问题。

iii) 包含关系

判别条件：若在 $O_1 (\epsilon_1, \delta_1, \Delta_1)$ 的 ϵ 域中存在 O_1 的逼近表示 M_1 ，在 $O_2 (\epsilon_2, \delta_2, \Delta_2)$ 的 ϵ 域中存在 O_2 的逼近表示 M_2 ，且 $M_1 \subset M_2$ ，则 $O_1 \subset O_2$ 。

误差调整：

O_1 的 ϵ 域为 O_1 、 O_2 的 ϵ 域交集的最大集； O_1 的 Δ 域为



O_1 、 O_2 的 Δ 域交集的最大集； O_1 的 δ 域延伸到刚超出 O_2 的 δ 域为止。

O_2 的所有容差不变。

iv) 相交关系

判别条件：若 $O_1(\varepsilon_1, \delta_1, \Delta_1)$ 与 $O_2(\varepsilon_2, \delta_2, \Delta_2)$ 既不重合又不相互包含，但它们的 ε 域相交，则 O_1 与 O_2 相交。

误差调整（相交元素，如交点、交线，容差表示构造）：

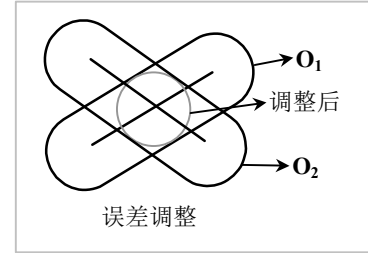
设相交所得的几何元素为 I ，则

I 的 ε 域为 O_1 、 O_2 的 ε 域交集的最大集；

I 的 δ 域为 O_1 、 O_2 的 δ 域交集的最大集；

I 的 Δ 域为 O_1 、 O_2 的 Δ 域交集的最大集；

O_1 、 O_2 的所有容差不变。



v) 模糊关系

判别条件：如果一个几何元素的 ε 域为空，或 δ 域不再包含在 Δ 域中，则该元素与其它元素之间的关系是模糊的，不确定的。

NOTE：正是因为有了容差表示的动态调整，才可能判别出模糊关系。

iii. 基于动态容差的几何关系所具备的性质

- 重合关系为等价关系。
- 包含关系具有传递性。如 $A \subset B$ 且 $B \subset C$ ，则 $A \subset C$ 。
- 如果 $A = B$ ，且 $A \neq C$ ，则 $B \neq C$
- 如果 $A \subset B$ ，且 $B \neq C$ ，则 $A \neq C$
- 如果 $A \subset B$ ，且 $A = C$ ，则 $C \subset B$
- 如果 $A \subset B$ ，且 $B = C$ ，则 $A \subset C$

iv. 模糊关系处理

当一个几何元素的 ε 域为空，或 δ 域不是 Δ 域的一个适当子集时，已无法用该容差表示判别其与其它几何元素之间的几何关系，也是显然不应该的。这说明该几何元素与某些几何元素之间具有矛盾的关系。

处理方法：当判别出模糊关系时，则调整初始误差，以消除模糊关系。一般为放大 τ ，以将不确定的关系都转变成重合、包含关系。

例如，将两个 $100 \times 100 \times 100$ 的立方体求并，其中一个为另一个沿着三个轴分别旋转一个角度 θ 的结果。

设 $\tau = 1.0e-3$ ， $\gamma = 1.0e-4$ ，则结果如下：

Angle	Result	After ambiguity handling
$\theta > 1.85e-6$	Intersecting	
$\theta = 1.8e-6$	Ambiguous	Coincident with $\tau \geq 4.0e-3$
$\theta = 1.5e-6$	Ambiguous	Coincident with $\tau \geq 3.0e-3$
$\theta = 1.0e-6$	Ambiguous	Coincident with $\tau \geq 2.0e-3$
$\theta < 0.9e-6$	Coincident	

v. 基于容差表示的布尔运算

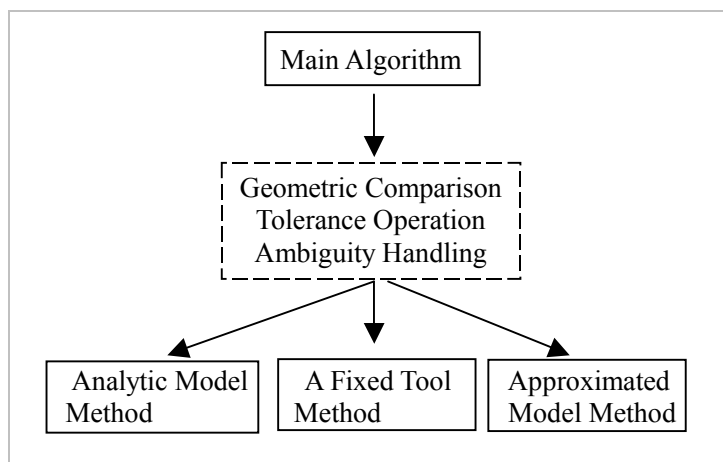
c) 对物体采用半空间与边界表示相结合的混合表示

$$obj = \bigcup_i \bigcap_j F_{ij}^+$$

半空间 (广义凸体的并)

d) 算法步骤

- i) 将 A、B 的半空间表示组合成一个新实体的半空间表示。
- ii) 基于半空间表示产生新实体的边、点表示。(借助容差表示的建立与几何关系判别)



- iii) 根据边、点以及半空间表示产生环、面等数据结构，得出新实体的边界表示。

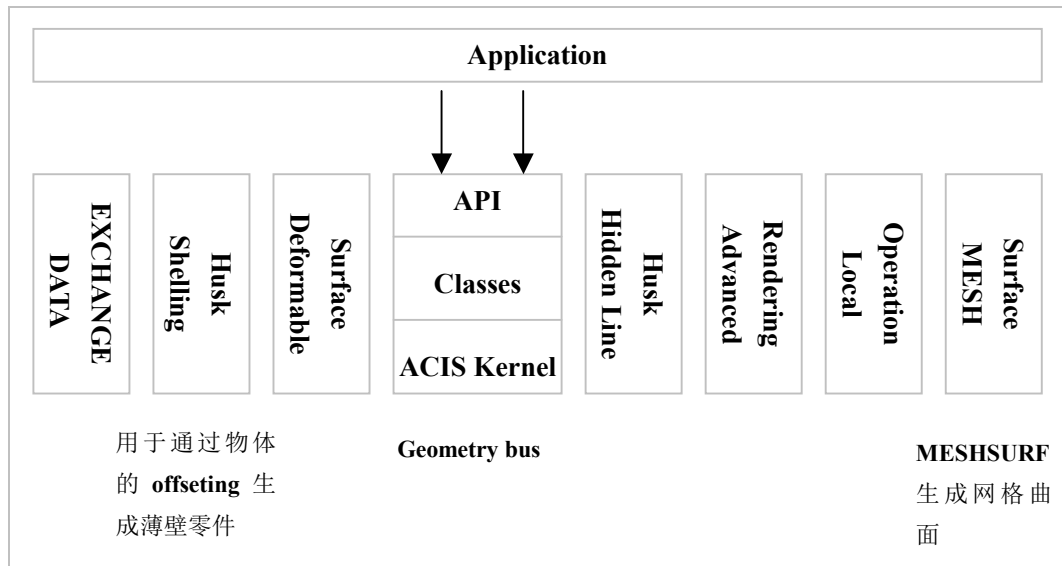
第十节 ACIS 几何造型系统介绍

1. 背景介绍

- ACIS 是一个具有灵活的 (flexible)、可扩张的 (extensible) 和开放的 (open) 结构的三维几何造型平台，目前最新版为 6.0。
- 美国 Spatial Technology 公司的产品，产生于 80 年代后期。ACIS 是由人名的第一个字母组成：Alan, Charles and Ian(即 I.C. Braid)。尤其是 I.C. Braid，曾主持开发过 BUILD、ROMULUS、PARASOLID 和 ACIS。
- 思想来源
 - i) CAD/CAM 系统种类多，如面向机械、面向建筑等，往往侧重点不同，与行业（汽车、传统机械、模具等）有关。许多大企业希望结合自己的行业知识开发自己专用的 CAD 系统。
 - ii) 所有三维 CAD 系统均需要有一个三维几何造型核心来支持。然而 robust 三维几何核心开发非常困难，而如果直接使用商品化系统做支撑，一方面过于庞大，一方面不易集成。而且太贵。
 - iii) 需要一个开放性好的三维几何造型平台。
- 用户
 - i) CAD 软件开发单位。
 - ii) CAD 研究单位，特别是高校中的 CAD/CAM 实验室。
 - iii) 中国大陆：北航、华中、华侨大学、浙大、南航；等。

- ACIS 采用面向对象的技术，C++语言开发。这在 CAD 产品中可能是第一家。

2. 系统结构（ACIS Architecture）

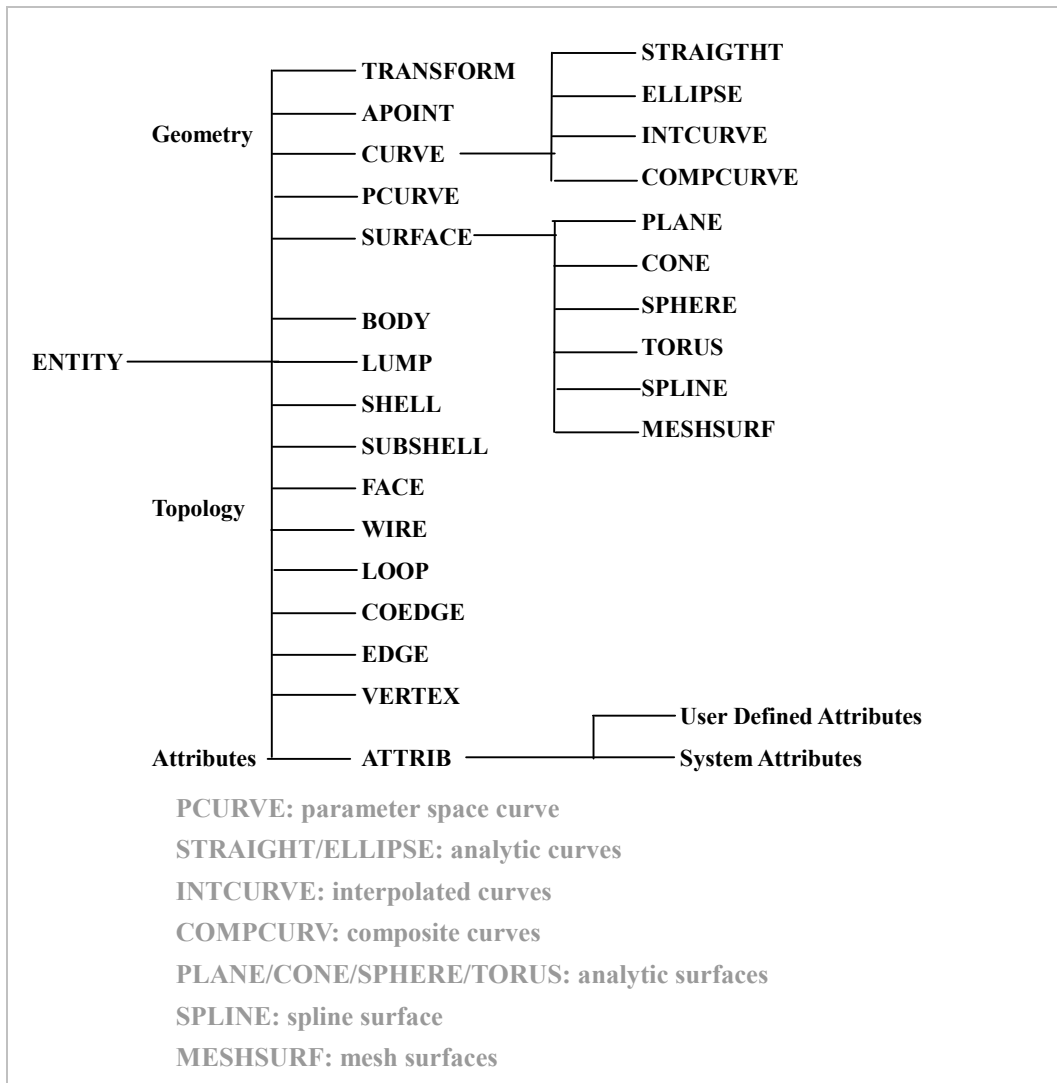


3. 数据结构

在所有的 .hxx 文件中描述。

3.1 ACIS 的类体系

分为三层。



3.2 ENTITY

- (1) 基类，提供 ACIS 的基本模型管理功能。如基于文件的输入、输出、复制、debug 等，因此这些功能放在基类中与应用程序见面，显然更有效。（应用程序编写简单；当增加新类时，应用程序不必改动。）
- (2) 虽然不是具体的几何、拓扑对象，但对保证 ACIS 的可扩充性、灵活性具有重要意义。

3.3 Advanced B-rep

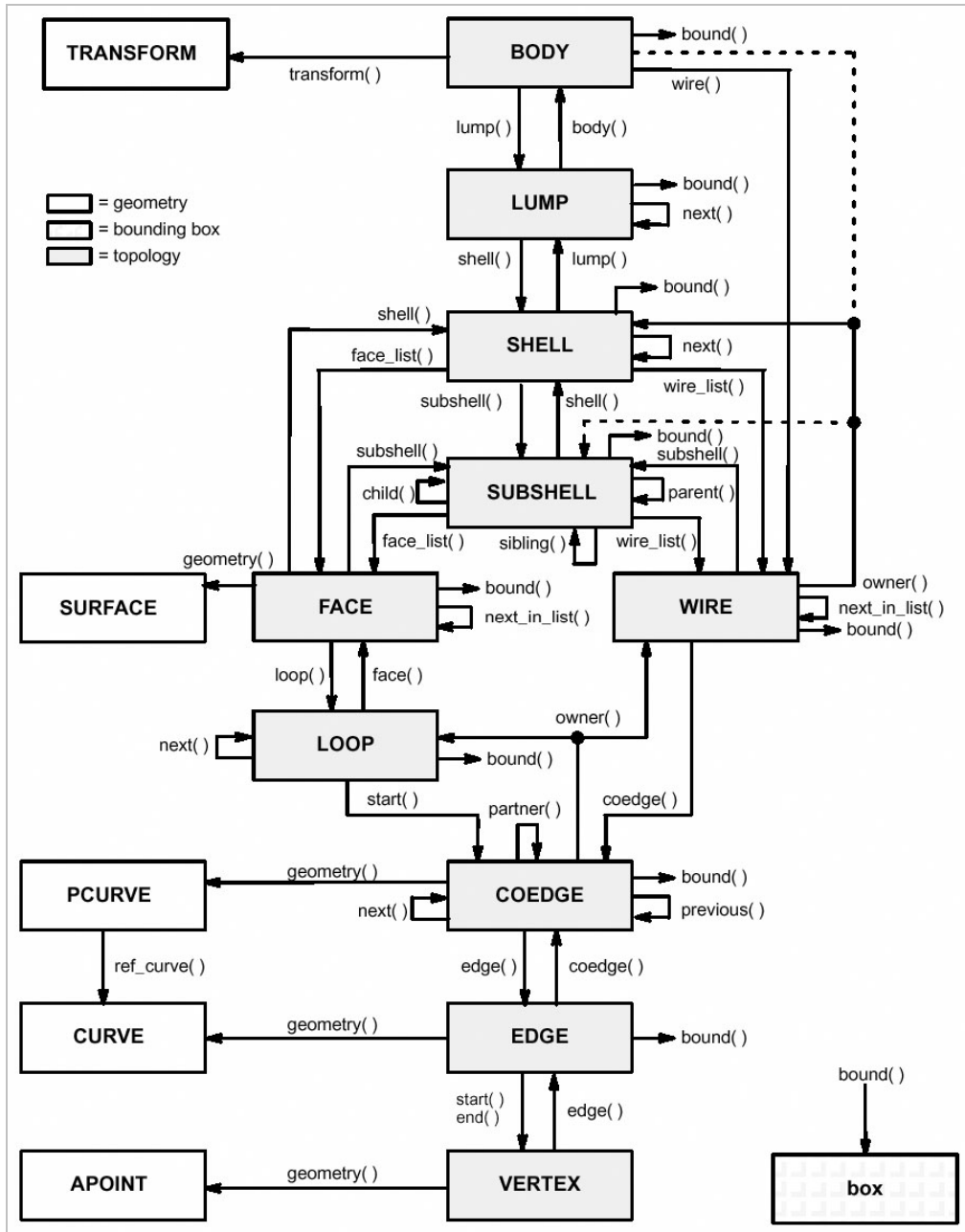


Fig. Implementation of Model Objects

上述 B-reps 表示是所见 B-reps 中先进性和实用性综合最优的边界表示。

系统提供了具体的数据结构文件 .hxx，存放在 kern6.0/kernel/kerndata 和 kerngeom 中，在编写应用程序是需 include 相关的 ACIS.hxx 文件。

先进性：

- (1) 线框、曲面、实体有机集成在一起
- (2) 能够表示非流形体和多维体

实用性：它本身已广为应用。

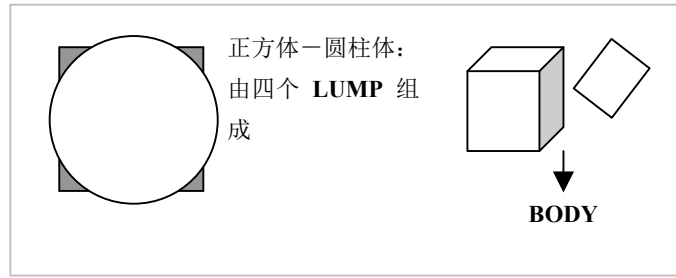
与传统半边数据结构的具体区别：

拓扑类：

BODY：最高层次的拓扑元素。

特点:

- 可以由一系列分离体组成 (LUMP 链表), 如右图, 正方体减去一个圆柱体后得到的物体由四个 LUMP 组成。
- 可以表示一个二维区域 (sheet, 具体的 sheet 可以是开的)。
- 可以表示线框 (称 wire body, 由 wire 的链表表示。线框可以是点、折线段 (开、闭) 和整个物体的线框模型)
- LUMP 和 WIRE 两者只有其一。



LUMP: 表示连通体或连通的二维区域。具体由多个 SHELL 链表表示, 可以是多个 SHELL。

SHELL: 表示 LUMP 的一个连通边界, 可以是二维的, 也可以是一维的。具体由一系列的 FACES 组成, 或由一系列 WIRES 组成。面链的尾巴是 NULL。

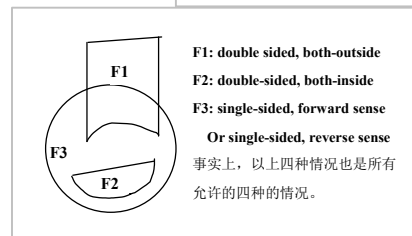
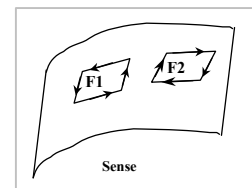
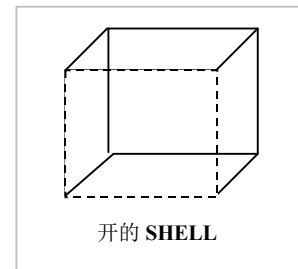
NOTE:

- SHELL 中的 FACE 指针与 WIRE 指针, 总有一个为 NULL。即表明其不能同时表示实体和线框, 原因是 coedge 无法兼顾这两者。
- SHELL 可以是开的, 可以是闭的。
- SHELL 可以是 complete 和 incomplete。
- 包含 incomplete shell 的 BODY 可能仅仅以其定义的边界区域 (而不是一个体) 参加造型操作, 甚至某些情况就不允许参加操作, 因仍会有二义性。

SUBSHELL: ACIS 将 SHELL 分解成层次结构的 SUBSHELLs 表示, 用以提高操作效率。由于该表示仅供系统内部使用, 因此如何分解, 如何使用均不得而知。(无 API, 也不能直接访问。)

FACE: 表示一张几何曲面的一部分 (应为连通区域)。具体由一系列 LOOPS 表示, 当 LOOP=NULL 时, 表示整张曲面, 可以是无界曲面。(LOOP 链的尾巴是 NULL)。

- sense(REVBIT): 表示 Face 的法向与 Surface 的法向相同还是相反。FORWARD 表示同向, REVERSE 表示反向。以 Glue 运算的结果为例。贴合面 $(A-B) \cup (B-A)$ 。其中面 $(A-B)$ 与面 $(B-A)$ 反向共面。
- sides(SIDESBIT): 表示 Face 为 single sided or double sided。单边面表示面的一边属于体内, 另一边属于体外; 双边面表示或两边同时属于体内, 或同时不属于体内。
- cont(CONTBIT): 表示双边面是 both-inside, 或 both-outside。前者表示体内面, 后者表示二维区域。



NOTE:

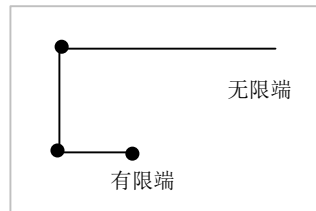
- 当 FACE 的 LOOP 指针为 NULL 时, FACE 仅含有几何信息 SURFACE, 而不再有拓扑信息, 从而等价于曲面模型, 此时许多实体造型操作将不能作用在 FACE 上。

ii) 由于引入了 sides 和 cont, 从而使得:

- sheet 仅需要一个面表示, 而以前均需用两个面表示 (不合理, 既浪费空间, 又会造成操作不可靠)。
- 能够表示体内面。

LOOP: 表示 FACE 的一个连通的边界。具体表示为一个 COEDGE 链。链可开、可闭, 故遍历时的结束条件 (等于 NULL 或等于 start edge)。

- 在 FACE 的所有环中, 部分内环, 外环, 因对某些曲面合适, 如圆柱。
- 对开的 LOOP, 开端可以是有限的, 也可以是无限制的。如果有一端是有限的, 则不完备, 如有一端是无限制的, 则为无限面。

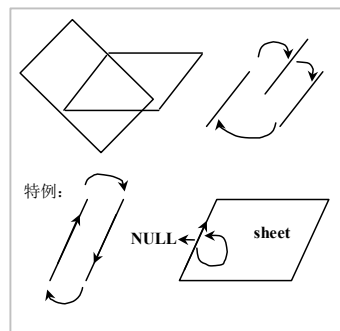


WIRE: 表示一系列连通的边, 这些边不与面相关。具体表示为 COEDGE 链。可开、可闭。(之所以不表示为 EDGE, 是因为 EDGE 本身不是链表。)

- 用于表示中心线、抽象元素等, 用于先生成物体的线框——>曲面——>实体。(ACIS 的 sweeping 操作的输入为 wire body)
- BODY—>WIRE—>COEDGE—>EDGE—>VERTEX 线框模型

COEDGE: 表示边在面中的引用, 仅与一个面相关。COEDGE 为双向链表。

- COEDGE 的 partner(partner_ptr)以逆时针方向构成一个循环链, 表示同一 EDGE 的所有 COEDGE。
- Sense: 相对于 EDGE 的方向, FORWARD 共享, REVERSE 反向。
- Pcurve: 指向 EDGE 在 COEDGE 所属面的参数域上的曲线。
- Start()、end() 返回 COEDGE 的起点、末点, 已考虑了 sense。



EDGE: 表示一条曲线上的一部分 (可以有界, 也可以是无界)。

- 具体由一个 curve 指针和两个端点指针表示。端点指针可以是 NULL, 端点为 NULL 表示边在那端不受到限制。如果曲线在那端是无限制的, 则 EDGE 在那端也是无限的。如果边和曲线是封闭的, 则或者两端点全部为 NULL, 或者两端点指向同一点。
- 曲线指针可以为 NULL, 此时必须两端点指向同一个 VERTEX, 用以表示孤立点。如圆锥的顶点。
- sense: 相对于 curve 的方向, FORWARD 为同向, REVERSE 为反向。

VERTEX: 除有几何点指针外, 包含与其相邻的一条边的指针, 指针可以为空, 此时表明该顶点包含一个 vertex ATTRIB, 其中包含多条边指针, 每一条边属于该点的一个不同的伞。

几何类: 从总体上分为两层, 模型的几何类 (大写表示) 和构造几何类 (小写表示)。前者是 ENTITY 的子类 (带有模型管理的函数、直接被拓扑元素所引用); 后者不是 ENTITY 的子类, 无 private member, 是通用的, 从而效率高。

TRANSFORM: transform 几何变换 4x4 矩阵。

APOINT: position x, y, z 坐标

Use-count: 该点被引用的数目, delete 该点时, 需对所有引用该点的 VERTEX 进行一致

性处理

CURVE (基类): curve 指针, curve 类, 用以表示曲线方程

Use-count: 该曲线被边引用的次数

STRAIGHT: straight: root_point_ptr(position), direction_ptr(unit_vector), equation

ELLIPSE: ellipse 指针 : centre_ptr(position), major_axis(vector),
normal(unit_vector) (其所在平面的法向), radius_ratio(real), equation

PCURVE (参数空间曲线):

use-count int

type int (PCURVE type=0; CURVE type!=0, 此时表示 wire body 中的曲线)

pcurve 的组成: par_cur 指针、sense(TRUE: indicates that the par_cur is reversed)

par_cur 的组成(总是依附于一个参数曲面): bs2_curve 指针、fit_tolerance(逼近误差)、
bs3_surface 指针 (指向参数曲线所属的样条曲面)

INTCURVE: intcurve 指针 (interpolated curve), 用于表示精确的 B 样条曲线, 或用于表示近似的交线。

Inter_curve 指针, 其组成如下:

- ✓ use_count
- ✓ bs3_curve
- ✓ fit_tolerance
- ✓ 两个 bs3_surface 指针, 表示曲线所在的曲面, 可以一个或两个均为 NULL。
- ✓ 两个 bs2_curve 的指针, 可以为 NULL。

COMPCURV: (composite curve) 一系列有序的, 首尾相连的曲线, 可以是解析曲线, 也可以是插值曲线。

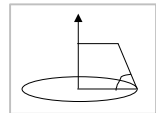
用于表示网格曲面生成的轮廓线或交线。

SURFACE (基类): 包含 use_count、surface 指针 (表示曲线方程)。当 use_count==0 时, 方可删除。

PLANE: plane: root_point(position), normal(unit_vector)

SPHERE: sphere: center(position), radius(double)

CONE: cone: ellipse: root_point、direction、major_axis、minor_axis、
radius_ratio、sine_angle(real)、cosine_angle(real)



TORUS: torus: center(position)、major_radius(double)、minor_radius(double)、
normal(unit_vector)

SPLINE: spline:→ spl_sur: use_count、bs3_surface

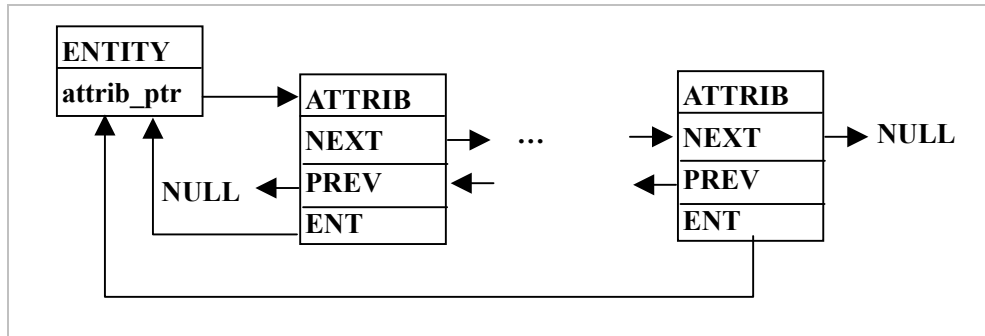
Sense

Surface const& equation()

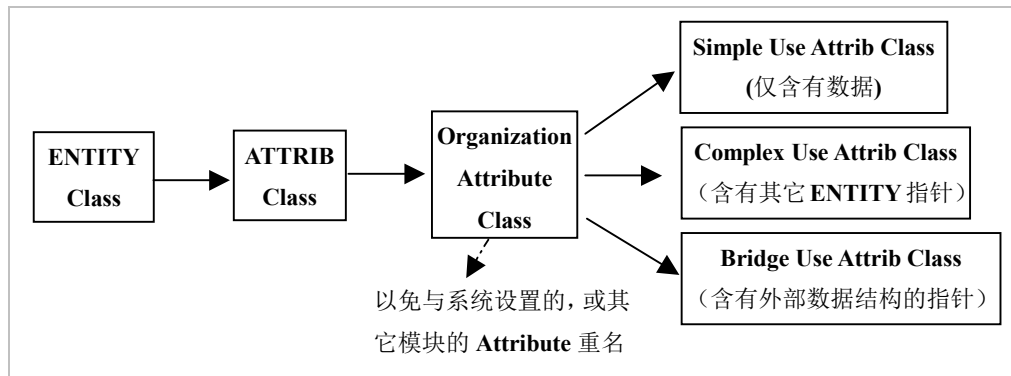
MESHSURF: meshsurf: 由一组 facets 构成, 用以表示散乱数据, 专用于 MESH SURFACE HUSK。

属性类: ATTRIB

横向联系:



纵向联系:



ATTRIB_STI、ATTRIB_TSL ← ACIS Kernel

FHL、PHL、CT 等 ← HUSKs

例如: PID(persistent ID)是系统赋给每个 ENTITY 的 attrib, 含 pid_base (session user id、session start time、在 session 内部的编号 Index、copy number)

用户 ATTRIB 类的定义:

att_org.hxx: 基于系统的宏 MASTER_ATTRIB_DECL (简单, 因具有固定格式)

att_org.cxx: MASTER_ATTRIB_DEFN

att_xxx.hxx: ATTRIB_FUNCTIONS(定义 att_xxx 类, 其为 att.org 的子类)

att_xxx.cxx: SAVE_DEF, RESTORE_DEF 等 (关键在于增加构造函数)

split_owner(), merge_owner(), trans_owner() 等, 保证在系统进行 Boolean 操作中生成新元素时, 能够自动生成新元素的有效 attribute。

系统提供给用户使用的通用 ATTRIB 类:

ATTRIB_GENERIC → ATTRIB_GEN_NAME → ATTRIB_GEN_INTEGER
ATTRIB_GEN_REAL
ATTRIB_GEN_STRING
ATTRIB_GEN_POSITION
ATTRIB_GEN_VECTOR
ATTRIB_GEN_POINTER
ATTRIB_GEN_ENTITY

定义用户自己的 ENTITY: 与定义 ATTRIB 类似

ENTITY_org. hxx	MASTER_ENTITY_DECL(...)
ENGITY_org. cxx	MASTER_ENTITY_DEFN(...)
ENTITY_xxx. hxx	ENTITY_FUNCTION(...)
ENTITY_xxx. cxx	ENTITY_DEF, SAVE_DEF...

定义自己的 API 函数：（可与 ACIS 的 API 同样使用）

api_xxx. hxx, api_xxx. cxx

3.4 系统容差变量

共三个。

1) resabs (resolution absolute)

缺省值为 10^{-6} 。用以表示系统中有意义的最小可表示量。当两点之间的距离小于 resabs 时，认为两点重合。

2) resnor (resolution normalized)

缺省值为 10^{-10} 。表示最小量与最大量之间的比值，即有效精度，最大量 10^4 。

NOTE：该值不需要变动，因其变动会导致造型操作的不可预测。

设计思想：double 为 14~16 位数，4 位为舍入误差，因此，10~12 位为有效区域。ACIS 取 10 位为有效表示范围。

3) resfit (resolution fit)

用作曲线、曲面的逼近容差，缺省值 10^{-3} 。含离散表示容差，交线逼近容差。

NOTE：resfit 应保持为 resabs 的 1000 倍。

NOTE：ACIS 本身不考虑单位，考虑到应用，缺省单位为 mm。如改变单位则 resabs 应作相应改变，但 resnor 不变。

4. Application Interfaces

1) C++ Interface

i) API functions

ii) C++ classes (ACIS)

iii) Direct Interface (DI) functions (不保证版本之间的一致性)

用户可以在自己的数据结构中直接引用 ACIS 的各种 ENTITY。

2) Scheme Interface

Scheme 是 ACIS 的解释性命令语言，是由 LISP 衍生出来的，用于支持快速、容易的、基于 ACIS 的应用开发。Scheme 语言可由用户扩充。

优点：运行速度快；不需要编译、连接。因此特别适宜于用户界面设计。

缺点：需要专门熟悉；不能方便地设计自己的数据结构。

3) MFC Interface

ACIS MFC(Microsoft Foundation Class)提供了与 MFC 的接口，因此用户可以在 ACIS 环境下利用 ACIS MFC 方便地进行窗口编程。